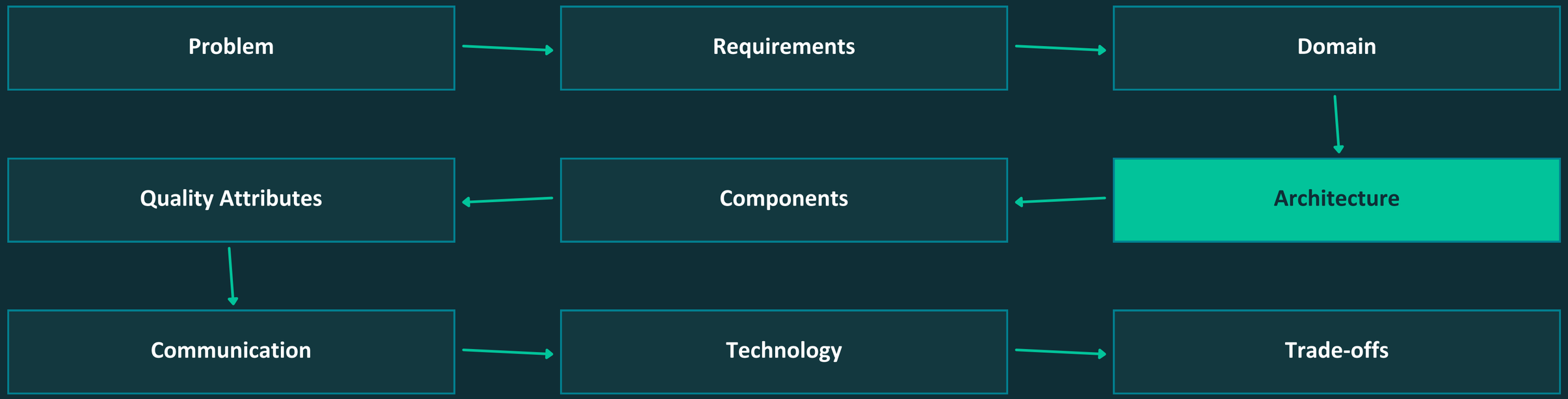
Software Architecture:
From Problem to System Design

A repeatable way of thinking for architecting a system.

Ravindu Deshan
Founder, Shuttlebook
Sr. Staff Software Engineer, CueGrowth.ai
AWS Certified SA-Associate

How architects think: the spine of this lecture



Architecture is a chain of reasoning**, not a diagram you draw at the end.**

ROADMAP

Where we are going today

1. **From SDLC to Software Architecture**
   Where architectural decisions happen in the life cycle
2. **Requirements: Functional & Non-functional**
   What we must know before deciding how to build
3. **From Problem Domain to System Design**
   Design around the problem, not the technology
4. **Design Principles: Cohesion, Coupling & SOLID**
   Organising code so change stays cheap
5. **Architecture: Monolith to Microservices**
   Styles, layers, communication, and what splitting costs
6. **Designing for Quality Attributes**
   Performance, scalability, reliability, security
7. **Putting It All Together**
   Architect a real system, end to end

You join a team in week 6. Which codebase do you want?

PROJECT A

```
project/
  app.js          (2,400 lines)
  helpers.js      (900 lines)
  stuff.js
  final_v3.js
  test_backup.js

Everything talks to
everything else.
```

PROJECT B

```
src/
  controllers/
  services/
  repositories/
  models/
tests/
docs/decisions/

Each part has one job.
```

Under 10%

of a typical modern application is code its own developers wrote.

The rest is frameworks, libraries and packages built by other people.
Knowing what to reuse, what to build yourself, and where to deploy is an architectural decision.

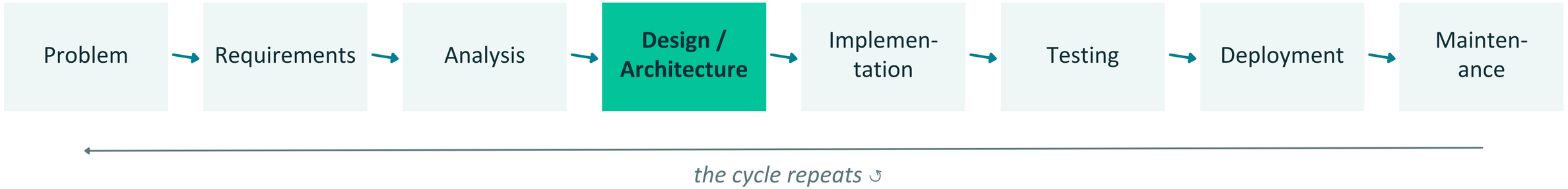
1

From SDLC to Software Architecture

Where does architecture actually happen in software development?

THE LIFE CYCLE

Where architecture lives in the SDLC



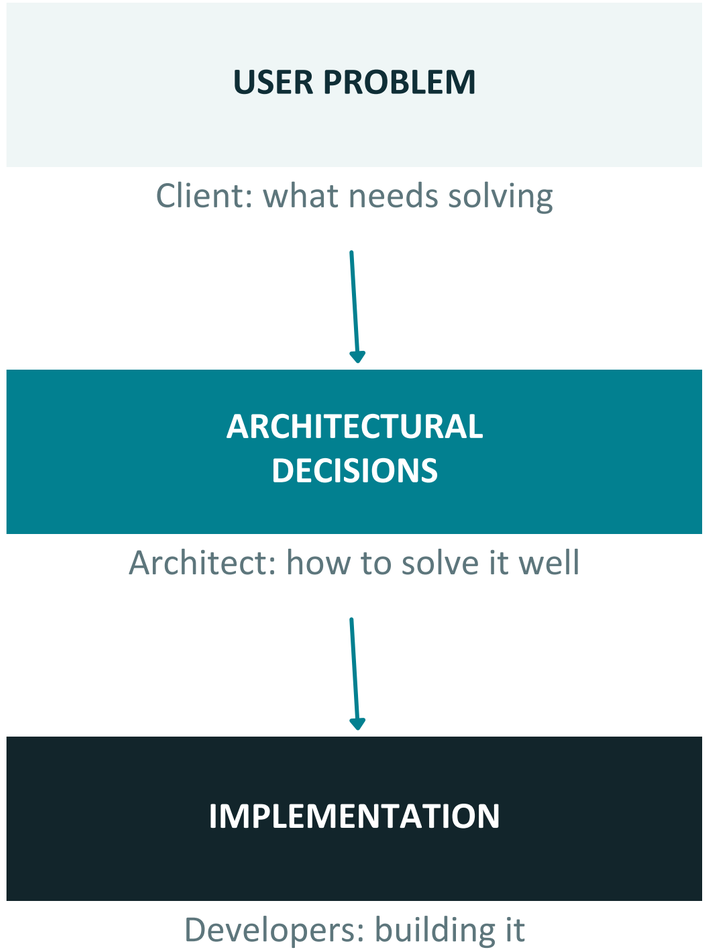
Architecture is not one isolated phase.

It begins when we start asking: "What kind of system do we need to build to solve this problem?" and architectural decisions continue to be made throughout implementation, testing and maintenance.

The highlighted box is where architecture is **most concentrated** , not where it is contained.

From user problem to implementation

Requirements & domain understanding flow down



Feedback from building flows back up

DEFINITION

Which of these are architectural decisions?

SQL or NoSQL for our data ?	Name of a private method ?
Button colour on the dashboard ?	Where the business rules live ?
List or dictionary for this variable ?	How the frontend talks to the backend ?
Which framework we build on ?	Whether this loop is a for or a while ?

Think: how expensive would this decision be to change in week 10?

DEFINITION

What software architecture actually is

Architecture is the set of decisions that are expensive to change later.

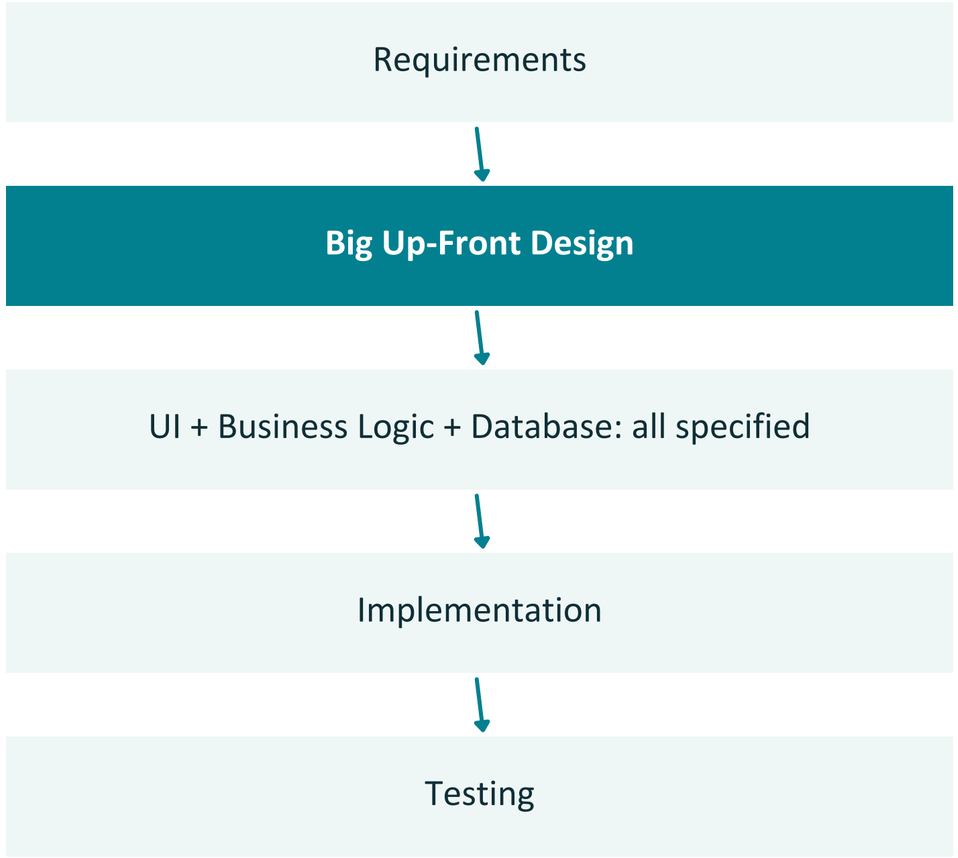
There is no universal best architecture, only better and worse fits for a given problem.

ARCHITECTURE	IMPLEMENTATION
SQL or NoSQL for our data	List or dictionary for this variable
Where the business rules live	Name of a private method
How the frontend talks to the backend	Whether this loop is a for or a while
Which framework we build on	Button colour on the dashboard

Architecture exists whether you plan it or not. **Unplanned just means accidental.**

THE TRADITIONAL APPROACH

Big up-front design: deciding everything first



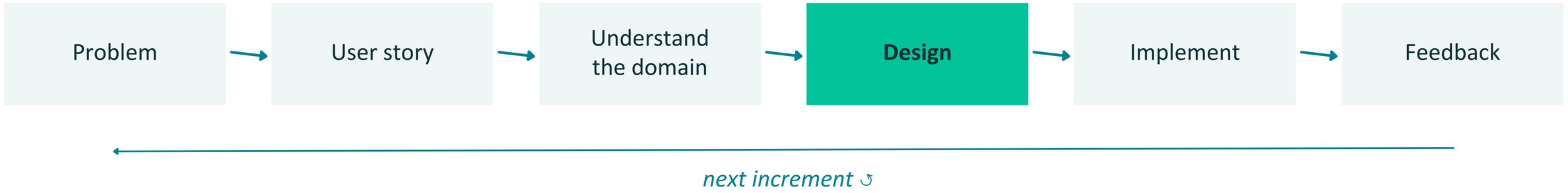
THE PROBLEM WITH THIS

- You make many architectural decisions before you have enough knowledge
- Users see nothing until the end; feedback arrives when change is most expensive
- Wrong assumptions are discovered in testing, not in week two

The failure mode is not planning; it is pretending early guesses are certain knowledge.

THE INCREMENTAL APPROACH

Design a little, learn a little, repeat



WHAT STAYS UP-FRONT

- The expensive-to-change decisions: language, framework, data model style, system boundaries

WHAT GETS DECIDED PER INCREMENT

- Everything else: endpoints, schemas, screens, internal structure, decided when you know enough

Modern architecture is not "don't design upfront." It is **"don't pretend you know everything upfront."**

There is no best architecture, only trade-offs

1. **Requirements**
   What must the system actually do, and how well must it do it?

2. **Team**
   Four people, mixed skills, limited weekly hours.

3. **Time**
   Twelve weeks including testing and a demo.

4. **Change**
   What is most likely to change in this project?

Every architectural choice buys you something and costs you something else.

Your job is not to avoid the cost. It is to know what you paid, and why.

2

Requirements: Functional & Non-functional

Before deciding how to build a system, what do we need to know?

TWO KINDS OF REQUIREMENTS

Functional and non-functional requirements

FUNCTIONAL: WHAT THE SYSTEM DOES

- User can register and log in
- Customer can place an order
- Admin can generate reports
- User can make a payment

NON-FUNCTIONAL: HOW IT SHOULD BEHAVE

- Response time under 2 seconds
- Support 10,000 concurrent users
- 99.9% availability
- Secure payment processing
- System must recover after failure

Functional requirements shape features. **Non-functional requirements shape architecture.**

WHY NFRS MATTER

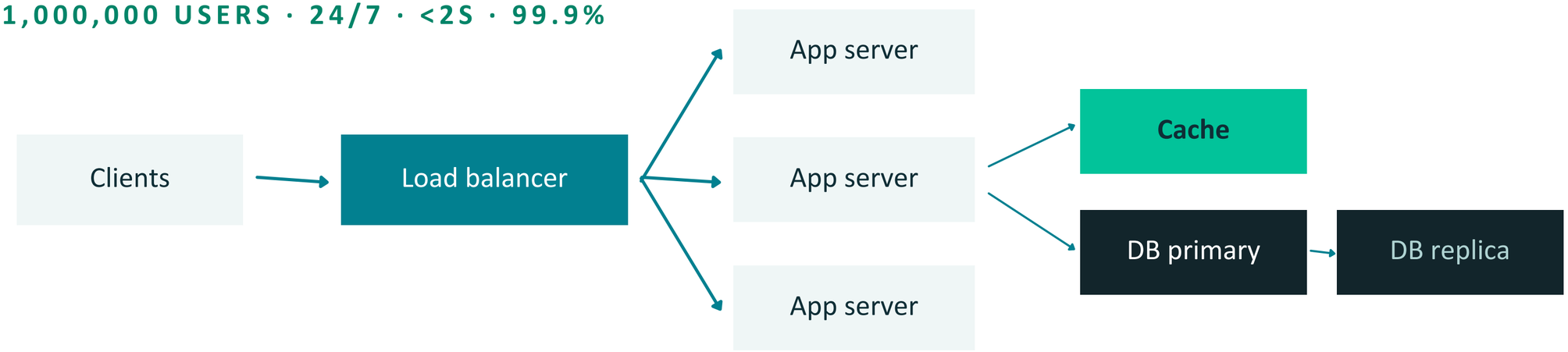
Same features, completely different systems

500 USERS · ONE CAMPUS



One machine can carry everything. Simple wins.

1,000,000 USERS · 24/7 · <2S · 99.9%



Identical features. The NFRs forced every new box on this diagram.

If someone asks "why is your architecture so complex?", the answer is always an NFR.

Architectural drivers: not all requirements are equal

"User can change their profile picture."

Probably does not drive architecture.
Any reasonable structure can absorb it.

"System must support 1 million users."

Absolutely does. It forces load balancing, stateless servers, caching, replication…



the few requirements that would force a different structure if they changed

A requirement is a driver if changing it would change your diagram. **Find those first.**

TWO-MINUTE EXERCISE

Spot the architectural drivers

1. Users can export a report as PDF
2. Handle 10,000 orders per minute during flash sales
3. Admin dashboard shows a pie chart of sales
4. A payment must never be recorded twice
5. Search results must appear in under 300 ms
6. The sign-up button text says "Join now"

Driver or feature? Decide each one, then defend it in one sentence.

Hint: ask "if this requirement doubled or vanished, would my diagram change?"

3

From Problem Domain to System Design

Design around the problem, not the technology

DESIGN AROUND THE PROBLEM DOMAIN

Where does design start: the stack, or the problem?

TECHNOLOGY-FIRST THINKING

- React
- C#
- .Net
- Node.js
- PostgreSQL
- Redis
- AWS
- Docker

A pile of tools, and still no idea what to build.

DOMAIN-FIRST THINKING, (ex:THE UNIVERSITY D(

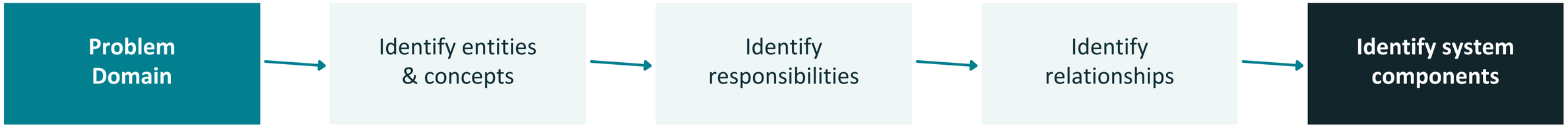
- Student
- Course
- Lecturer
- Enrollment
- Assignment
- Examination
- Payment

These concepts exist in every implementation.

The structure of your software should reflect **the structure of the problem you are solving.**

THE METHOD

From problem domain to system components



Problem Domain	Identify entities & concepts	Identify responsibilities	Identify relationships	Identify system components
Online university	Student · Course · Enrollment · Payment	"Enrolment checks prerequisites & capacity"	Student enrols in→ Course, via Enrollment	Enrollment module · Payment module …

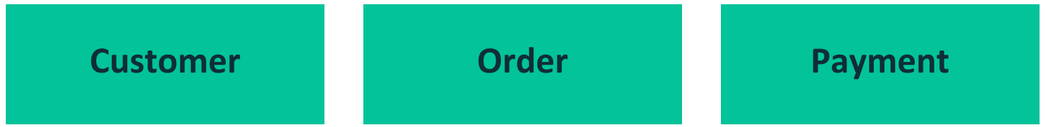
WHY THIS WORKS

Requirements talk about the domain ("a student enrols in a course"). Code lives in components. This pipeline is the translation between them, and it gives every component a name your client already understands.

Name your components after the problem, and the code explains itself.

AN IMPORTANT NUANCE

Domain concepts do not automatically become services



three important domain concepts

OPTION A: MODULES IN ONE MONOLITH



one deployable app

OPTION B: THREE MICROSERVICES



Both are valid designs for the same domain. Which one is right depends on the trade-offs; that is the rest of this lecture.

The architecture should reflect the important concepts and boundaries of the domain**, not mirror every noun as a service.**

EXERCISE · 5 MINUTES

Design a food delivery system

In your project teams, identify the major concepts of this problem domain:

1. **Major users**

   Who interacts with the system?

2. **Major entities**

   What "things" does it manage?

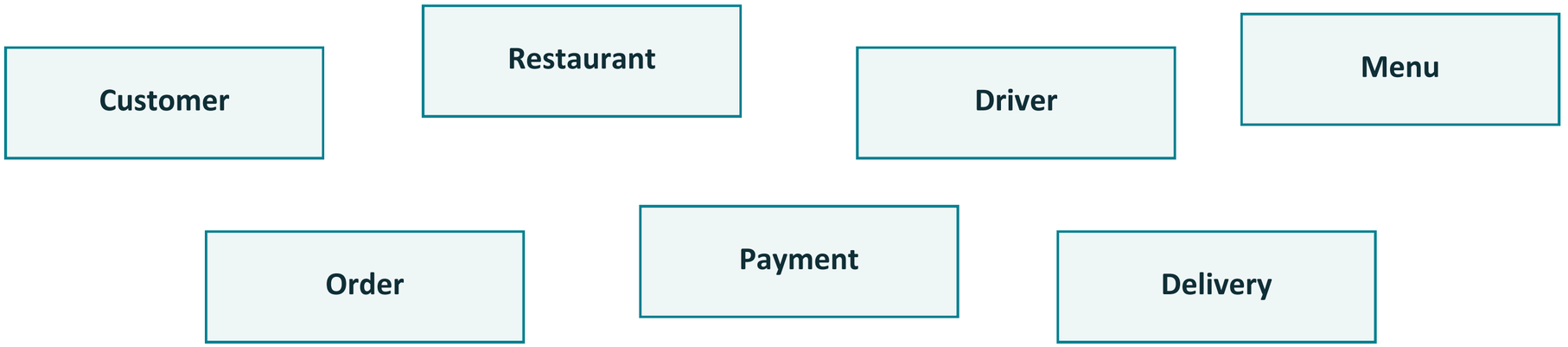
3. **Major actions**

   What happens in it?

4. **Business areas**

   What clusters of activity exist?

The concepts you (probably) found



"If these are the major concepts in the problem, shouldn't our software structure somehow reflect them?"

Give every concept a job and its data, and modules appear

Concept	Responsibility: what does it do?	Data only it owns
Order	accepts orders, enforces order rules, tracks status	`orders, order items, status history`
Payment	charges cards, refunds, records every money movement	`transactions, refunds`
Delivery	assigns drivers, tracks the trip to the door	`assignments, GPS trail`
Menu	what each restaurant sells, at what price, when	`menus, items, prices, availability`
Customer	who is ordering, where they live, how to reach them	`profiles, addresses`

DOMAIN → DESIGN · ONE CONCEPT UNFOLDED

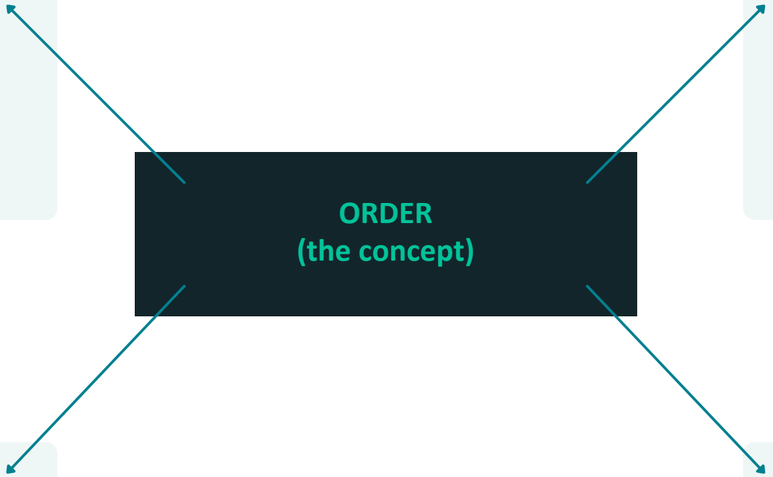
Everything "Order" generates, before you write any code

1 · COMPONENT

OrderService module
(today a folder, someday maybe a service)

2 · DATA

```
orders(id, customerId,
  restaurantId, status,
  total, placedAt)
```



3 · API · RESPONSIBILITIES AS VERBS

```
POST  /orders
GET   /orders/42
PATCH /orders/42/status
```

4 · INTERACTIONS

calls Payment (sync: answer needed)
notifies Restaurant (async: queue)
emits OrderPlaced for whoever cares

Nobody "designed an API" here. **It was hiding inside the domain analysis all along.**

4

Design Principles: Cohesion, Coupling & SOLID

Organising components so change stays cheap

CORE IDEA

Coupling and cohesion

COHESION

How strongly related the things inside one unit are.

High cohesion means one module does one job well.

COUPLING

How much one unit depends on another.

Low coupling means you can change one thing without breaking five others.

Keep related things together. Depend on as little as possible.

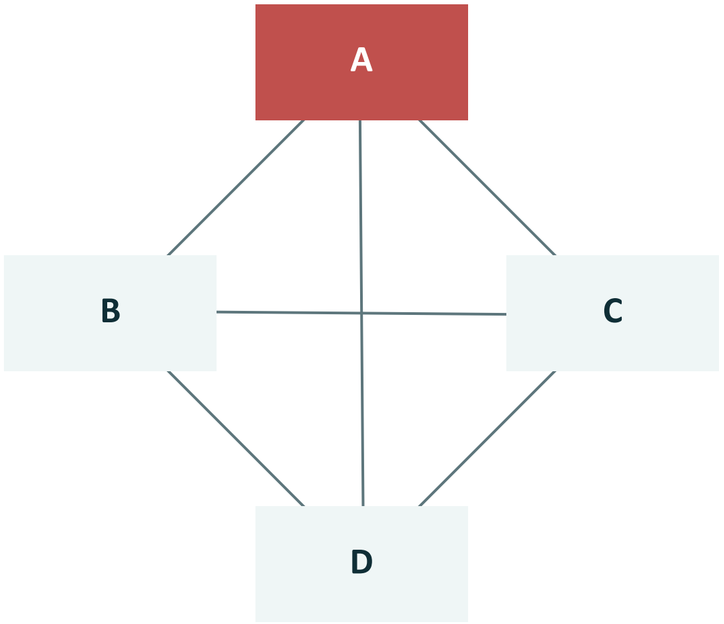
WHY THIS MATTERS IN EVEN A SMALL PROJECT

Coupling is the reason two teammates cannot work in parallel. It shows up as merge conflicts, as "your change broke my feature", and as one person becoming the only human who understands a file.

COUPLING, VISUALIZED

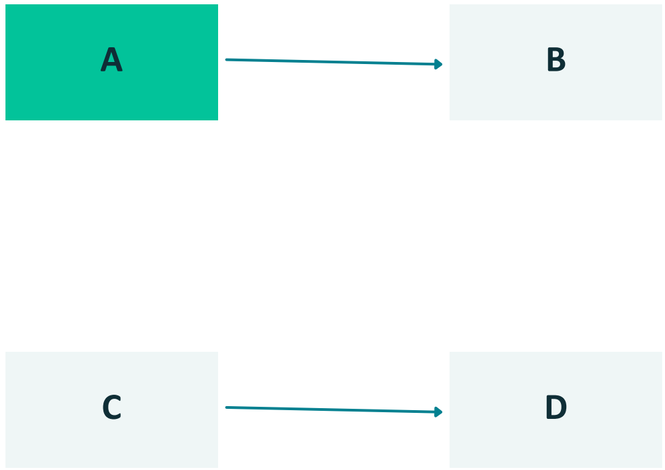
What a small change does in each system

TIGHTLY COUPLED



Change A → B, C and D all break.
Every edge is a reason to have a meeting.

LOOSELY COUPLED



Change A → only B might notice,
and only through a stable interface.

Count the arrows. Every arrow is a way for a change to spread.

COHESION, VISUALIZED

One class doing six jobs vs six classes doing one

LOW COHESION: SIX UNRELATED JOBS

```
class UserService {
  login()
  createInvoice()
  sendEmail()
  calculateTax()
  generateReport()
  processPayment()
}
```

HIGH COHESION: ONE JOB EACH

```
AuthenticationService
```

```
PaymentService
```

```
InvoiceService
```

```
EmailService
```

```
ReportingService
```

Good architecture aims for **high cohesion and low coupling.**

ANTI-EXAMPLE

One class, four jobs, endless pain

```
class Student {
  constructor(name, email) { ... }

  save() {                    // knows SQL
    db.query("INSERT INTO students ...");
  }

  sendWelcomeEmail() {        // knows the email
provider
    sendgrid.send({ to: this.email, ... });
  }

  toHtml() {                  // knows the UI
    return `<div class="card">${this.name}</div>`;
  }
}
```

ASK YOURSELF

- The email provider changes. How many files do you touch?
- You want to test the registration rule. Do you need a database?
- Two teammates edit this file in the same week. What happens at merge time?
- How many reasons does this class have to change?

THE FIX

One reason to change, per unit

```
class Student {              // just the data and rules
  constructor(name, email) { ... }
}

class StudentRepository {    // only this knows SQL
  save(student) { ... }
}

class EmailService {         // only this knows SendGrid
  sendWelcome(student) { ... }
}

class StudentView {          // only this knows HTML
  render(student) { ... }
}
```

WHAT YOU GAINED

- Swap the email provider by editing one file
- Test the rules without a database
- Four teammates can work in four files
- A new person can find things by name

PRINCIPLES

SOLID, weighted by how much it will help you now

S	**Single Responsibility**	One class, one reason to change. The one you will break daily.
O	**Open / Closed**	New behaviour should be added by extension, without constantly editing tested code.
L	**Liskov Substitution**	A subtype must behave correctly anywhere its parent type is expected.
I	**Interface Segregation**	Many small interfaces beat one fat one; clients should not depend on what they don't use.
D	**Dependency Inversion**	High-level business logic should not depend on low-level details. Your gateway to layers and testing.

Treat these as habits that reduce coupling, not as rules to obey.

Single Responsibility: one job, one reason to change

REAL-WORLD ANALOGY

A restaurant where one person takes orders, cooks, delivers and does the billing works when it is quiet. At lunch rush, everything fails at once.

Specialised roles = the kitchen scales.

ONE CLASS, FIVE JOBS

```
class OrderManager {          // 5 reasons to change
  placeOrder()       // ordering rules
  calcDiscount()     // pricing rules
  chargeCard()       // payment provider
  sendInvoiceEmail()// email provider
  logAnalytics()     // tracking tool
}
```

✗ VAT rule changes → you edit the same file as payments

`OrderService` `PricingService` `PaymentGateway` `InvoiceMailer` `AnalyticsClient`

✓ each box: one job, one reason to change, one owner in your team

The question that finds violations: **"how many different reasons does this class have to change?"**

Open/Closed: add behaviour without editing tested code

✗ MODIFICATION: REOPEN AND RETEST

```
function pay(order, type) {
  if (type === "card")   { ... }
  else if (type === "paypal") { ... }
  else if (type === "crypto") { ... }
  // every new method edits
  // this tested function
}
```

✓ EXTENSION: ADD A CLASS, TOUCH NOTHING

```
interface PaymentMethod {
  pay(order)
}
class CardPayment   implements ...
class PayPalPayment implements ...
class CryptoPayment implements ...  // NEW FILE ONLY
```

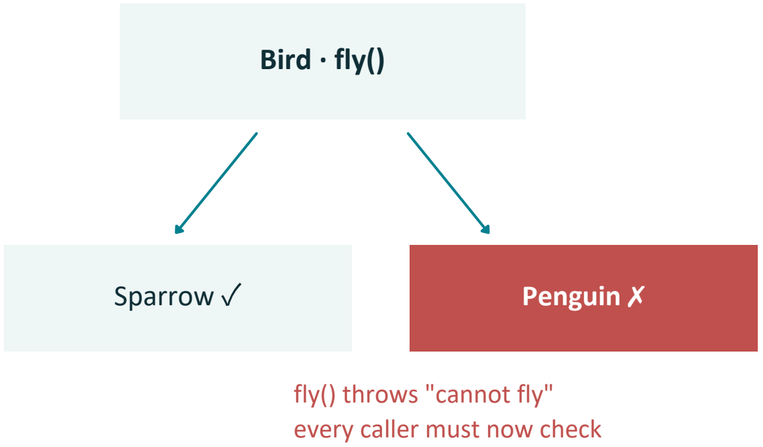
REAL-WORLD ANALOGY

Your phone: you install new apps every week, but you never open the phone and re-solder its board. The platform is closed for modification, open for extension.

Design the stable core once, then let new behaviour arrive as new files.

Liskov Substitution: a subclass must not lie

THE CLASSIC: A PROMISE THE SUBCLASS CANNOT KEEP



fly() throws "cannot fly"
every caller must now check

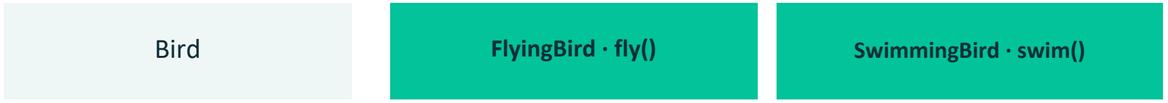
PROJECT VERSION OF THE SAME BUG

```
// looks fine, until COD arrives
class Payment { refundToCard() }
class CardPayment extends Payment ✓
class CashOnDelivery extends Payment {
  refundToCard() { throw Error(); }
}

refund(payment)  // crashes for COD
```

Fix: Refundable interface, or refund() that returns a result instead of throwing.

THE FIX: SPLIT THE PROMISE



Interface Segregation: no client forced to depend on what it does not use

✗ ONE FAT INTERFACE

```
interface OfficeMachine {
  print()  scan()
  fax()    staple()
}

class SimplePrinter implements OfficeMachine {
  print() { ... }        // the only real one
  fax()   { throw ...; } // forced stub
  staple(){ throw ...; } // forced stub
}
```

✓ SMALL, HONEST CONTRACTS

Printable · print()

Scannable · scan()

Faxable · fax()

SimplePrinter → Printable

OfficeBeast → all three

Each class implements only the contracts it can honestly keep.

In your project: if the driver app must implement restaurant callbacks it never uses, every driver release now depends on restaurant changes. Split the interface, decouple the teams.

Many small interfaces beat one fat one: nobody signs a contract they cannot keep.

DEPENDENCY INVERSION

Depend on a promise, not on a product

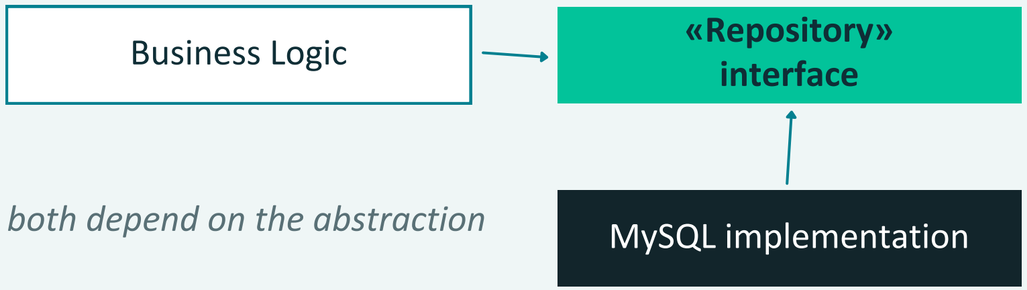
TIGHTLY COUPLED

```
class RegistrationService {
  constructor() {
    this.db = new MySqlDatabase();
    // locked in forever
  }
}
```

INVERTED

```
class RegistrationService {
  constructor(repository) {
    this.repo = repository;
    // any repository works
  }
}
```

THE ARROWS FLIP



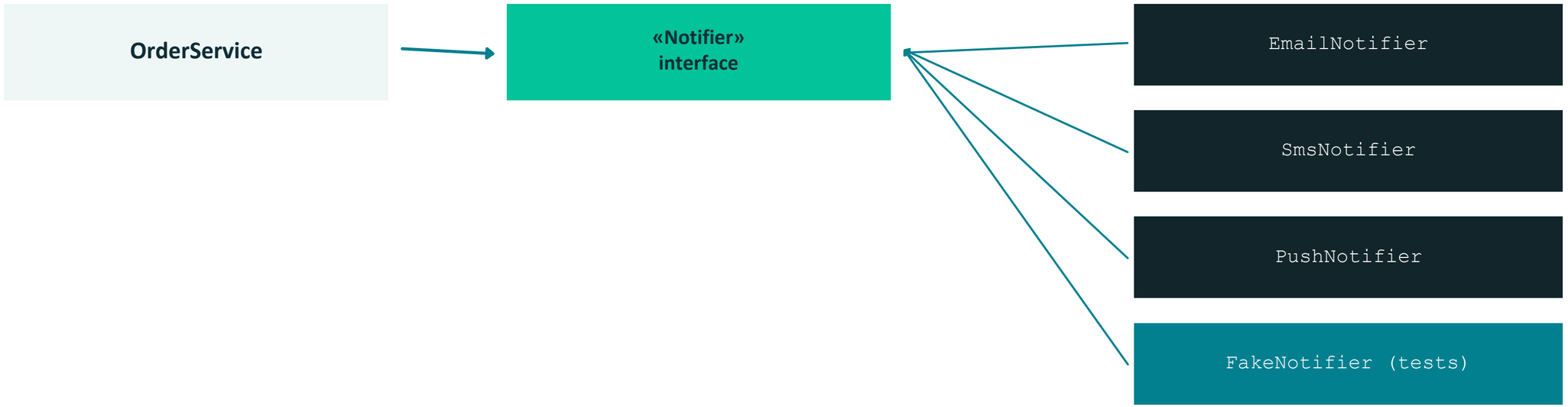
both depend on the abstraction

WHAT THIS ONE CHANGE BUYS YOU

```
// production
new RegistrationService(new MySqlRepository());
// fast tests, no database
new RegistrationService(new
InMemoryRepository());
// swap later, one line
new RegistrationService(new
PostgresRepository());
```

Leads to: dependency injection · clean architecture · hexagonal architecture, etc

Dependency Inversion at work: swap the world, keep the logic



implementations point at the interface: the arrow is inverted

Real story: provider rate-limits you before the demo. Swap = one line. No DIP: edit every file that sends email.

Analogy: appliances plug into a standard wall socket. The house never rewires for a new kettle.

High-level policy owns the interface. Low-level detail obeys it. That is the inversion.

SOLID · RECAP

Five principles, one enemy: coupling

S	**Single Responsibility** One reason to change per class	→ Find the class with the most imports. Split it.
O	**Open/Closed** New behaviour = new code, not edits	→ Find the longest if/else chain. Introduce an interface.
L	**Liskov Substitution** Subclasses keep the parent's promises	→ Find a method that throws "not supported". Redesign it.
I	**Interface Segregation** No forced dependencies on unused methods	→ Find a stubbed method that only throws. Split the contract.
D	**Dependency Inversion** Depend on interfaces, not products	→ Find a "new Database()" inside business logic. Inject it.

This week, each teammate fixes ONE violation. **SOLID is a habit, not a definition.**

Do not over-engineer

ABSTRACTION HAS A COST

Fifteen interfaces and a factory for a to-do list app is a worse crime than none. Every layer of indirection is another file a teammate must read before they can help you.

ADD ABSTRACTION WHEN IT HURTS

Feel the coupling first, then remove it. Two implementations existing today is a reason for an interface. One hypothetical future one is not.

YAGNI: "You Aren't Gonna Need It"

Build for the requirements you have, in a way that does not block the requirements you might get.

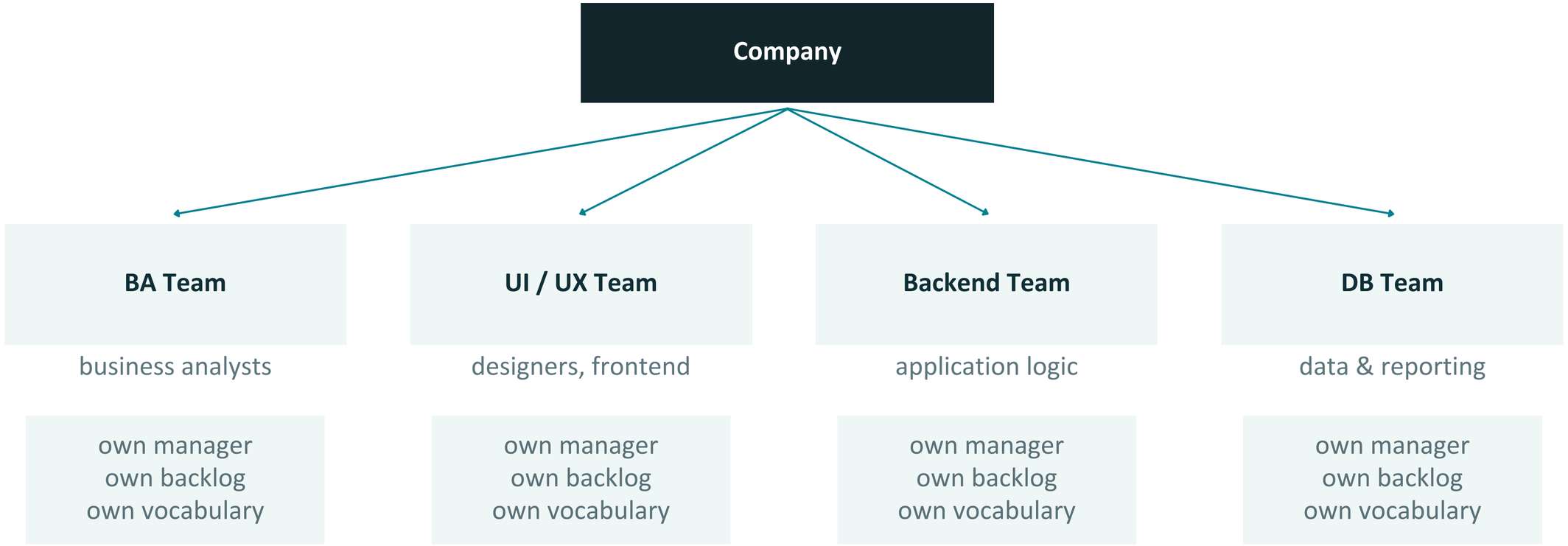
5

Architecture: Monolith to Microservices

Styles, layers, how systems talk and and what splitting really costs

PEOPLE FIRST

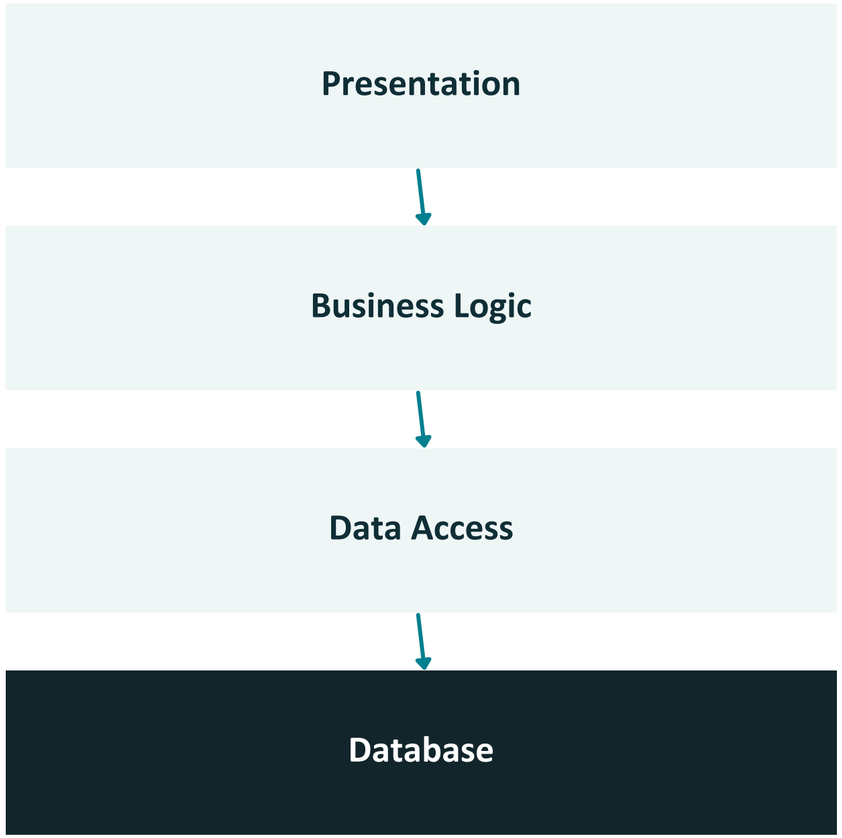
How organisations traditionally structure themselves



Functional silos: each team is excellent at one specialty, and and every feature needs all four.

A CLASSIC STYLE

N-tier architecture: organised in horizontal layers



screens, pages, apps: what users see

the rules of the system

queries, persistence

the single source of truth

dependencies point downward only

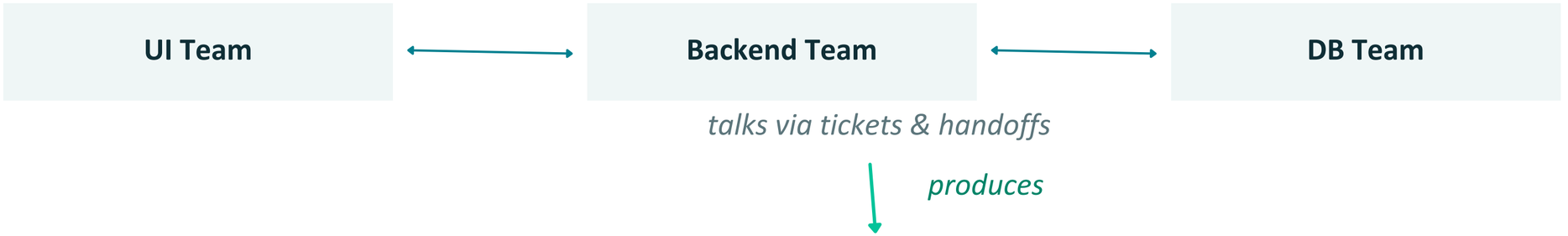
One style among many, common wherever teams specialise by function. Not a law of nature.

WHY STRUCTURE LEAKS

Conway's Law (1968)

> *"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."*
>
> **Melvin E. Conway, "How Do Committees Invent?", Datamation, 1968**

THE ORGANISATION



THE SYSTEM



System boundaries appear where communication is weakest. Choose your team structure like you choose your architecture.

THE MODERN COUNTER-MOVE

Cross-functional teams own business capabilities

Team A

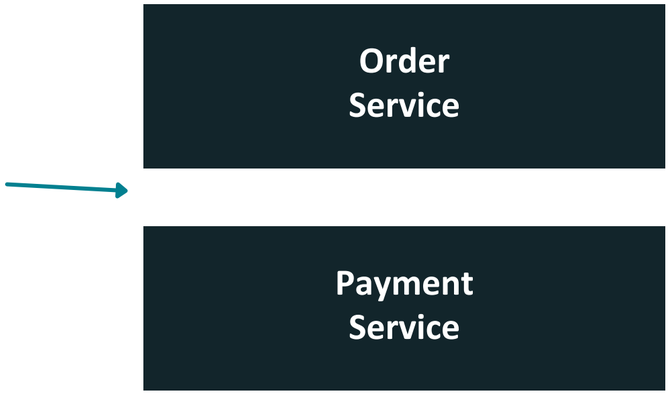
owns Ordering

- Backend
- Frontend
- QA
- Domain knowledge

Team B

owns Payments

- Backend
- Frontend
- QA
- Domain knowledge



each capability independently owned, built and shipped

Organisational structure can influence architectural structure. Microservices are not a cure for silos; they are what capability-owning teams naturally produce.

INSIDE THE SERVER

Three layers, one direction of dependency

1

Controller

Receives HTTP, validates input, returns status codes. This is where the API lives.

Knows HTTP. Knows nothing about SQL.

↓

2

Service

The rules of your system. "A student cannot join a full course."

Knows neither HTTP nor SQL.

↓

3

Repository

Reads and writes the database. The only place SQL appears.

Knows SQL. Knows nothing about HTTP.

THE DEPENDENCY RULE

Arrows point one way only.

Controllers call services. Services call repositories. Never backwards. Never skipping a layer. The UI never touches the database.

This is single responsibility and dependency inversion, applied at project scale.

One request, end to end

1. Browser sends POST /courses/42/registrations
2. Framework routes it to the registration controller
3. Controller reads the body and calls the service
4. **Service applies the rule: is the course full?**
5. Repository inserts a row into the database
6. Service returns the new registration object
7. Controller replies 201 Created with JSON

NOTICE

The rule at step 4 lives in exactly one place.

It works the same whether the request came from a browser, a mobile app, a command-line tool, or an automated test.

That is the whole payoff of layering.

DAY ONE IMPLEMENTATION DECISION

What folders do I actually create?

PACKAGE BY LAYER

```
src/
  controllers/
    courseController.js
    studentController.js
  services/
    registrationService.js
  repositories/
    courseRepository.js
  models/
  config/
```

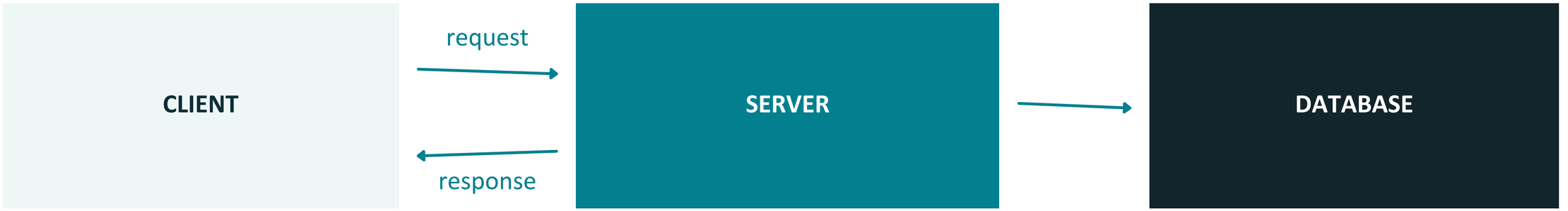
PACKAGE BY FEATURE

```
src/
  registration/
    controller.js
    service.js
    repository.js
  courses/
    controller.js
    ...
  shared/
```

By feature usually wins for group projects: one teammate owns a folder. Either is fine; no structure is the only wrong answer.

HOW SYSTEMS TALK

Client and server: the fundamental split



Browser or mobile app
Presentation only

Business rules
Security and validation

The single source of truth

WHY SPLIT AT ALL

- Many clients share one server
- Rules and data live in one trusted place
- Each side is built and deployed separately
- Frontend and backend work split across the team
- The network between them is now part of your design

The last bullet is the important one: the network is about to charge us for everything.

THE PROTOCOL

Anatomy of a request and a response

REQUEST

```
POST /courses/42/registrations HTTP/1.1
Host: api.campus.lk
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{ "studentId": 7 }
```

RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 991,
  "studentId": 7,
  "courseId": 42 }
```

Method
what you want to do

URL
which resource

Headers
who you are, what format

Body
the data itself

Status
what happened

REST

Resources are nouns. Verbs are already provided.

VERB	URL	MEANING	RETRY SAFE?
GET	`/courses`	List all courses	Safe, repeatable
GET	`/courses/42`	Read one course	Safe, repeatable
POST	`/courses/42/registrations`	Create a registration	Not repeatable
PUT	`/students/7`	Replace a student record	Repeatable
PATCH	`/students/7`	Update part of a student	Repeatable
DELETE	`/registrations/991`	Remove a registration	Repeatable

Idempotency in one line: a double click on POST creates two registrations. On PUT or DELETE it does not.

API DESIGN

Nouns in the URL, verbs in the method

SMELL

```
GET  /getCourse?id=42
POST /createRegistration
POST /deleteStudentById
GET  /updateCourseName

The method carries no meaning.
Every endpoint is a surprise.
```

CONVENTIONAL

```
GET    /courses/42
POST   /courses/42/registrations
DELETE /students/7
PATCH  /courses/42

Predictable without reading
the backend code.
```

REST is a set of conventions, not a technology.

Its real value on your project: your teammate can guess your endpoint correctly on the first try.

RESPONSES

Status codes: three families, a few members

2xx — **It worked**

- 200 OK
- 201 Created
- 204 No Content

4xx — **You messed up**

- 400 Bad Request
- 401 Unauthenticated
- 403 Forbidden
- 404 Not Found

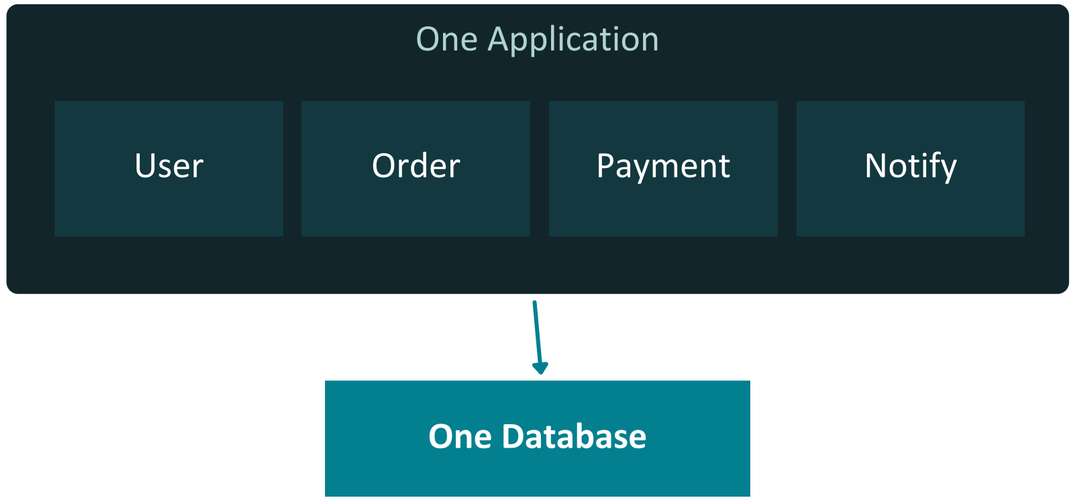
5xx — **We messed up**

- 500 Internal Error
- 503 Unavailable

Returning 200 with an error message inside the body is the most common API mistake students make.

STYLE 1

The monolith: one application, one deployment



ADVANTAGES

- Simple to develop, deploy, debug locally
- Simple transactions: one database
- Low operational complexity

DISADVANTAGES

- Codebase grows; coupling can creep in
- Scale and deploy everything together
- Coordination hurts as the team grows

For a team of four with twelve weeks: this column of advantages is worth a lot of marks.

NOT A BINARY CHOICE

Monolith → modular monolith → microservices

1

Monolith

One deployable unit. All code together. Layers may be messy or clean.

Where most projects start

2

Modular monolith

Still one deployment, but internal boundaries are clean enough to separate later.

Where your project should live

3

Microservices

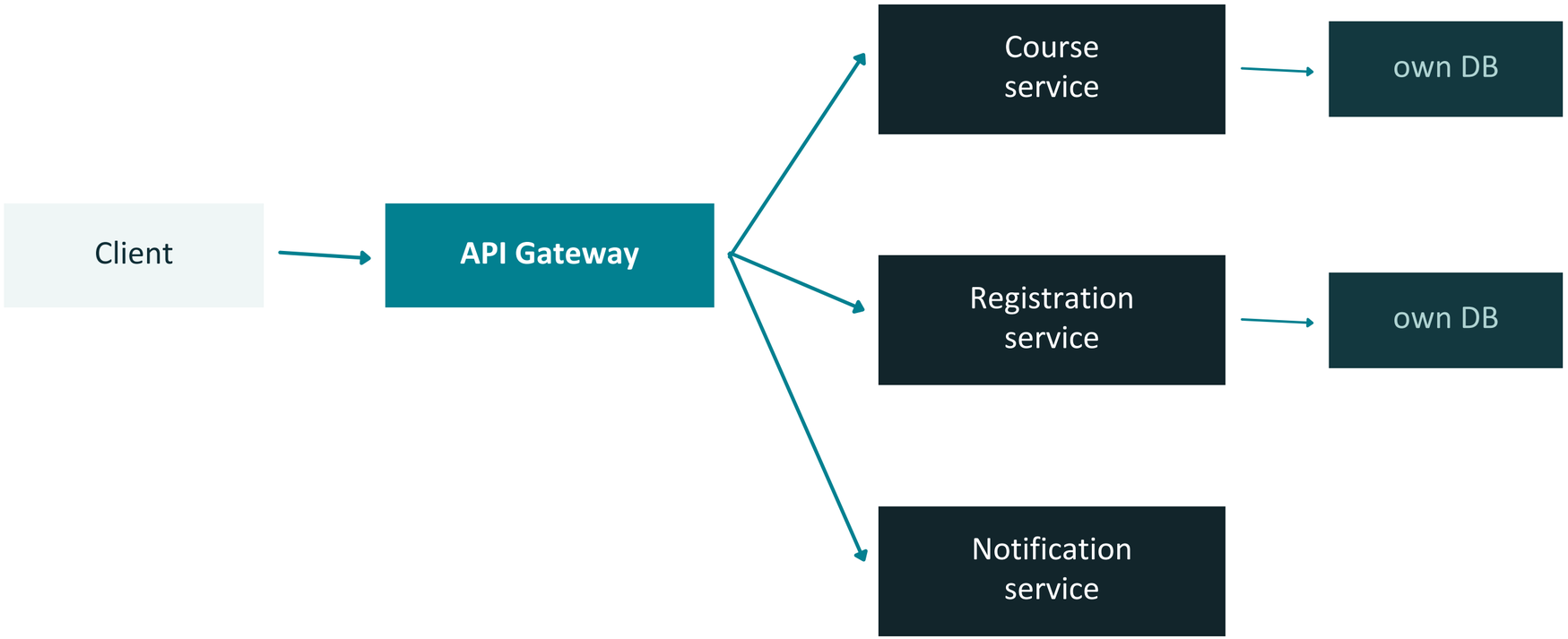
Each module is its own deployed service with its own database, talking over the network.

Where large organisations end up

A well-structured monolith can become services later. **A tangled monolith cannot become anything.**

STYLE 2

Microservices: independently deployed, talking over the network



services also call each other; each service is a client of the others

GAINS

- Independent deploys
- Independent scaling
- Failure isolation
- Team autonomy

COSTS

- Network complexity
- Distributed failures
- Data consistency
- More infrastructure

THIS IS WHY WE LEARNED REST FIRST: every arrow above is an HTTP call (or a queue message).

DEFINITION, PRECISELY

What exactly is a microservice architecture?

A style where the system is a set of small, independently deployable services, each owning **one business capability** and its own data, communicating over the network.

✓	**Independently deployable**	ship it alone, no coordination meeting	*Can you release it on a Tuesday by yourselves?*
✓	**Scoped to a business capability**	payments, ordering, delivery: nouns the business knows	*Would a manager recognise its name?*
✓	**Owns its data**	its tables are private; others use its API	*Does any other service read your tables? Must be no.*
✓	**Communicates over the network**	REST, gRPC or events; never shared memory	*Is every dependency an explicit call or event?*
✓	**Owned end to end by one team**	build it, run it, get paged for it	*Does exactly one team own its on-call?*

Fail any one of the five and you have a distributed monolith: **network prices, no independence.**

THE FORCES

What actually pushes an organisation to split

1 **Release traffic jam**

Two hundred developers, one deployment pipeline. Every release waits for everyone.

2 **Uneven load**

One module gets hammered, the rest are idle, but you must scale the whole thing together.

3 **Blast radius**

One memory leak in one module takes the entire product offline.

4 **Team autonomy**

Teams want to ship on their own schedule with their own technology choices.

Three of these four are people problems, not technical ones.

THE HONEST CHECKLIST

Do you actually need microservices?

✓ SIGNALS YOU NEED THEM

- Several teams block each other's releases every week
- One component needs 10x the scaling of the rest
- A part genuinely needs different technology
- You can run 24/7 monitoring, CI/CD and on-call per service
- Domain boundaries are stable and well understood

✗ SIGNALS YOU DO NOT

- One team that fits around one table
- The domain is still changing every sprint
- No dedicated ops or platform experience
- The product has not met real users yet
- You want it because big companies use it

Score your project honestly. **More ticks on the right: build the modular monolith.**

WORKED EXAMPLE

Same checklist, two systems: decide for each

SYSTEM A · "CAMPUSEATS" UNIVERSITY FOOD ORDERING

Team	4 students, 12 weeks
Users	3,000 students, one campus
Load	~2 orders per minute at peak
Domain	still changing every sprint
Ops	nobody on call, no budget

VERDICT: modular monolith · one deployment · one database

SYSTEM B · "ISLANDWIDE" NATIONAL DELIVERY PLATFORM

Team	9 teams, 60 engineers
Users	2 million, nationwide
Load	ordering spikes 20x at lunch
Deploys	payments needs same-day hotfixes
Ops	24/7 platform + on-call teams

VERDICT: microservices · drivers: spike isolation, deploy autonomy

Checklist rows that flipped the answer: team count · load shape · deploy independence · ops maturity. Nothing else mattered.

Same checklist, opposite verdicts. **The system decides, not the fashion.**

WORKED EXAMPLE · CONTINUED

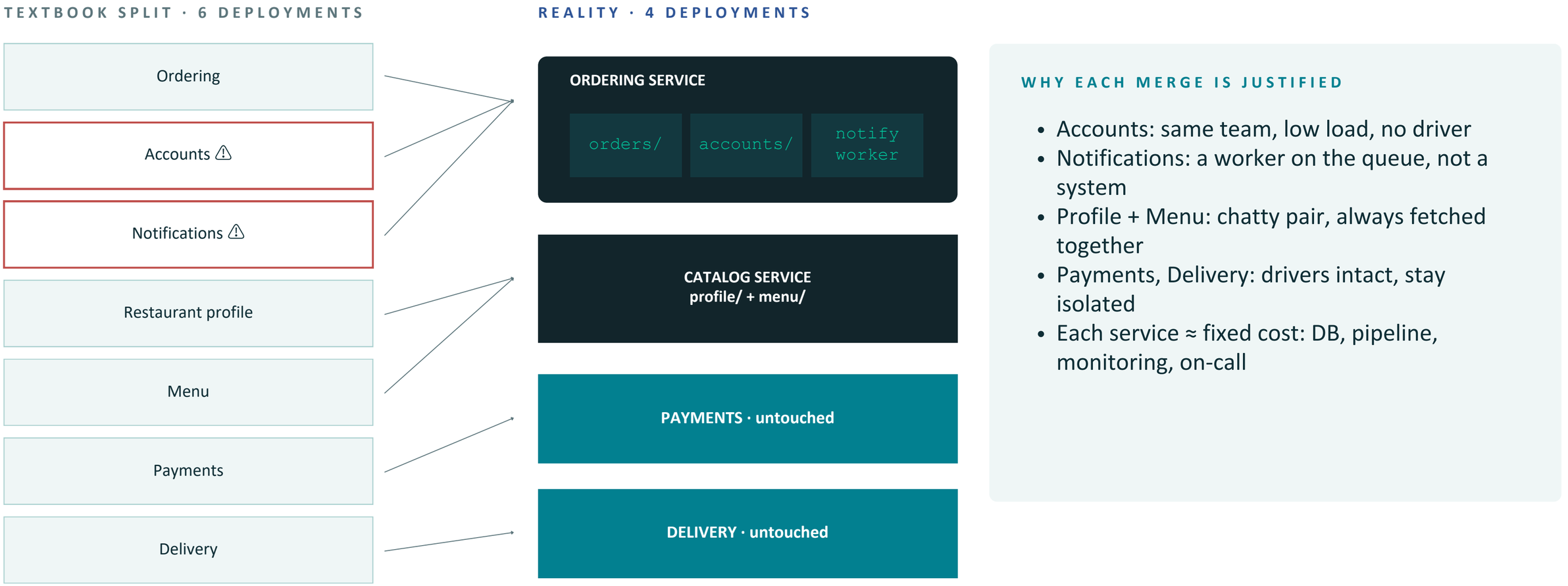
IslandWide: which capabilities earn a service?

Ordering	own service	20x lunch spike: must scale alone
Payments	own service	compliance + same-day hotfixes: isolate it
Delivery / dispatch	own service	GPS ping load shape + one team owns it
Restaurant & menu (Catalog)	own service	read-heavy, cache-friendly, rare deploys
Accounts / profiles	⚠ candidate	no driver of its own: low load, stable
Notifications	⚠ candidate	no driver: a queue worker, not a system

The discipline: **no driver, no service.** The two ⚠ rows are resolved on the next slide.

WORKED EXAMPLE · THE REALITY PASS

Deviating from the textbook: merge where boundaries cost more than they earn



WHY EACH MERGE IS JUSTIFIED

- Accounts: same team, low load, no driver
- Notifications: a worker on the queue, not a system
- Profile + Menu: chatty pair, always fetched together
- Payments, Delivery: drivers intact, stay isolated
- Each service ≈ fixed cost: DB, pipeline, monitoring, on-call

6 deployments → 4, roughly a third off the infra bill. **Merging without a lost boundary is architecture, not defeat.**

THE HONEST PART

What the network charges you

1. **The network fails**

   A method call becomes a request that can time out, arrive twice, or vanish. Every call site now needs a failure plan.

2. **No shared transaction**

   "Registration saved but notification lost" becomes a design problem. Consistency becomes eventual, and you must handle it.

3. **Operational weight**

   Many deployments, service discovery, configuration per service, debugging across scattered logs.

4. **Boundaries become permanent**

   Moving a responsibility between services is far harder than moving a class between folders.

A distributed mess is far worse than a monolithic mess; you cannot even read it in one place.

MICROSERVICES IN PRACTICE

The Six Challenges

1 Debugging across services

One request = logs in 6 places. Needs correlation IDs + tracing.

2 Testing

Integration tests need other services alive. Contract tests help.

3 Data consistency

No joins, no shared transactions. Eventual consistency is on you.

4 API versioning

Your change breaks their old client. Now you run v1 and v2.

5 Operations load

Every service: its own pipeline, monitoring, alerts, secrets.

6 Local development

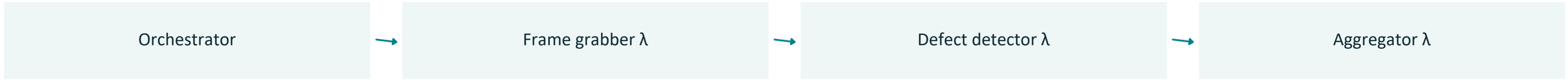
Day-one teammate must run 12 containers just to log in.

None of these kill the idea. All of them cost real hours. Budget for them or stay monolithic.

REAL WORLD · THE RETURN TRIP

Prime Video, 2023: microservices back into a monolith

BEFORE: DISTRIBUTED COMPONENTS



✗ every hop = orchestration cost + data transfer between components

AFTER: ONE PROCESS



✓ same steps, now function calls in one process: no network, no per-hop state

≈ 90% cost reduction and it scaled further

Also: Segment, 2018: "Goodbye Microservices": collapsed ~140 services back into fewer, bigger ones for the same reason: overhead outgrew the benefit.

Every boundary has a price. **Pay it only where it earns its keep.**

Microservices are not an upgrade. They are a trade-off.

"You shouldn't start a new project with microservices, even if you're sure your application will be big enough to make it worthwhile."

Martin Fowler, "MonolithFirst" (martinfowler.com, 2015)

IF YOU DO SPLIT

Five microservice patterns worth knowing

1 API Gateway

One front door. Clients call the gateway; it routes to services.

```
Client → Gateway → Services
```

2 Database per Service

Each service owns its data. No shared tables, ever.

```
Order svc → Order DB
```

3 Service Discovery

Services find each other by name through a registry, not hard-coded IPs.

```
Who is "payments"? → Registry
```

4 Circuit Breaker

Stop calling a service that keeps failing; fail fast to a fallback.

```
A → B ✗ → fallback
```

5 Async Messaging

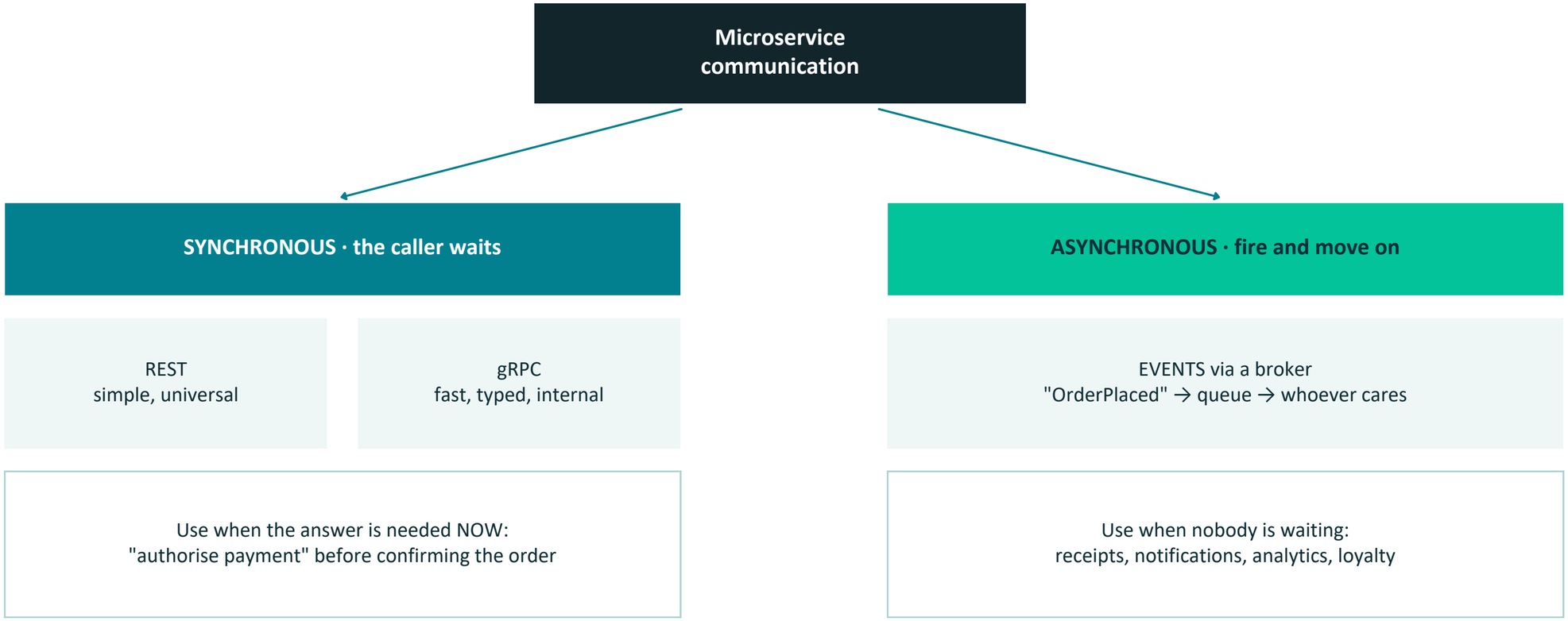
Send a message to a broker and move on; the receiver processes it later.

```
A → Queue → B
```

There are many more microservice patterns. These five are the ones relevant to the architecture you are about to design.

Patterns are vocabulary, not obligations. Name them in your report only where they earn their keep.

How services talk: two branches, one tree



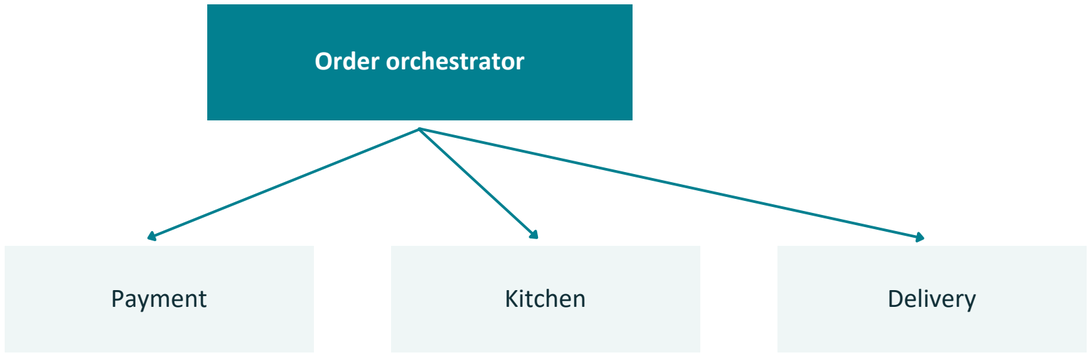
Brief note: a system built ENTIRELY around events as its backbone is Event-Driven Architecture: a separate architectural style, advanced courses. Here we only borrow the event.

Per interaction, ask one question: is anyone waiting for this answer?

COORDINATING A WORKFLOW

Orchestration vs choreography: who owns the flow?

ORCHESTRATION · A CONDUCTOR



✓ flow visible in one place, clear error handling
✗ conductor knows and couples to everyone

CHOREOGRAPHY · A DANCE



each service reacts to the previous event:
OrderPlaced → PaymentTaken → MealReady → ...

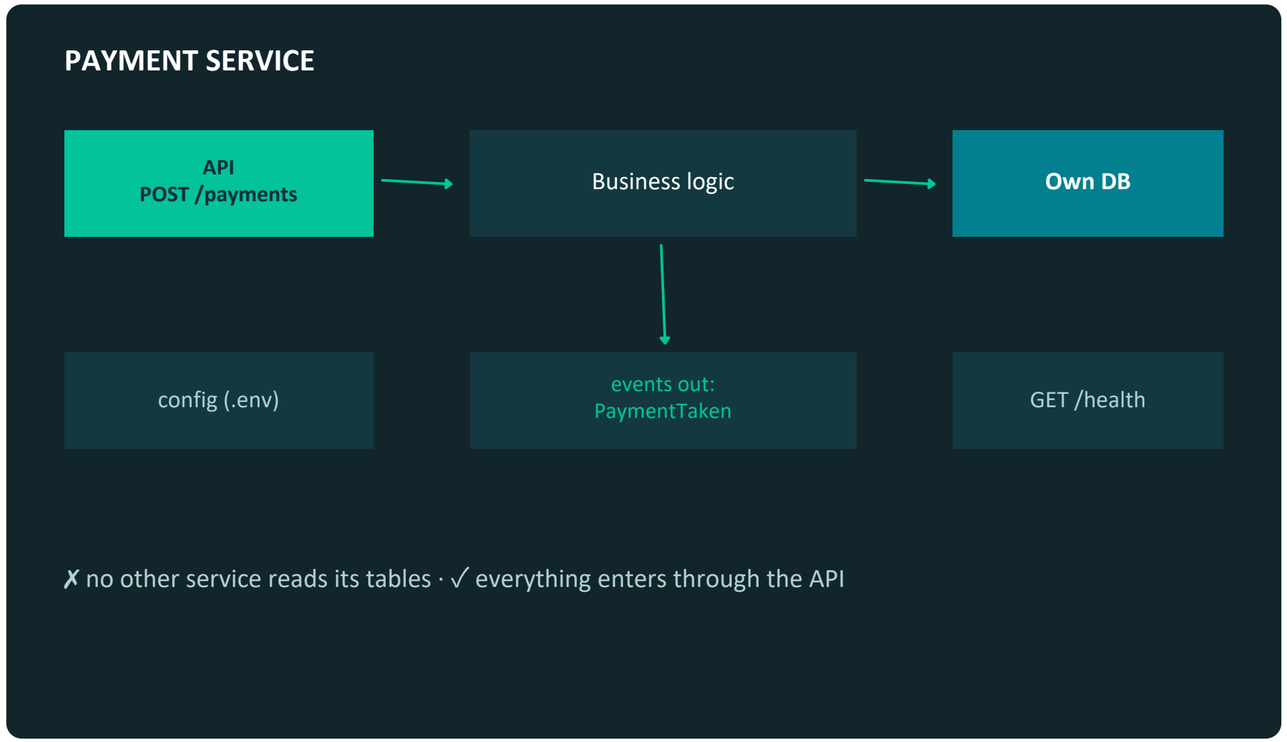
✓ no central coupling, add steps freely
✗ nobody sees the whole flow, errors scatter

Rule of thumb: orchestrate money and critical workflows (payments need one clear owner of failure). Choreograph broadcasts (notify, analyze, reward). The pattern name for distributed workflows: saga.

Ask of every workflow: who needs to see it, and who owns its failures?

PRACTICAL DESIGN

Anatomy of one well-designed microservice



1. Pick one business capability (payments, not "utils")
2. Define the API contract FIRST, then build in parallel
3. Own your data: your tables are private
4. Choose edges: sync where waiting, async where not
5. Plan failure: timeouts, retries, idempotency

The acid test: **can you deploy it alone, on a Tuesday, without a meeting?**

SIZING

How big is "micro"? Capability-sized, not code-sized

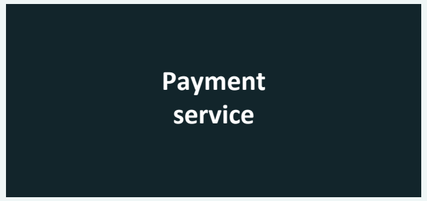
✗ Nano-services



One function per service:
AddTax svc, FormatDate svc ...

Network cost > business value.
Ops load explodes.

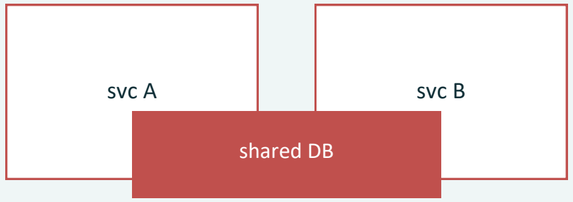
✓ Capability-sized



Payment service: everything
payments, nothing else

One team, one reason to change,
deploys alone.

✗ Distributed monolith



Services that share a DB and
must deploy together

Network tax + coupling kept.
Worse than a monolith.

Split along capabilities the business would recognise. Never along lines of code.

6

Designing for Quality Attributes

How architecture answers the non-functional requirements

PERFORMANCE | SCALABILITY | RELIABILITY | SECURITY

YOU ALREADY USE ALL OF THIS

Quality attributes hiding in your daily apps

Google search feels instant	→ Caching + replication across the planet	PERFORMANCE
WhatsApp ticks: sent ✓ delivered ✓✓	→ Async message queues between phones	ASYNC
Netflix at 8 pm never buffers	→ Popular titles cached inside your ISP (CDN)	PERFORMANCE
A bank transfer never half-happens	→ Transactions: all or nothing	RELIABILITY
Flash sales survive the stampede	→ Load balancer + many stateless servers	SCALABILITY
You stay logged in on every device	→ Tokens, not server memory	STATELESS

Nothing in this section is exotic. You are a daily user of every idea we are about to design with.

QUALITY 1 · PERFORMANCE

What is Performance?

Latency and throughput are different things

LATENCY: ONE OPERATION



How long does one user wait?

THROUGHPUT: THE WHOLE SYSTEM

1,000 requests / second

How much work per unit of time?

Low latency ≠ high throughput.

A bus moves fifty people an hour (great throughput) but each waits at every stop (poor latency). A sports car is instant for one person and useless for a crowd. Architecture constantly trades one for the other; batching, for example, buys throughput with latency.

Why performance is money: Amazon measured ~1% of sales lost per extra 100 ms of latency; Google measured ~20% less traffic when results took 500 ms longer. *(widely-cited industry experiments, 2006–09)*

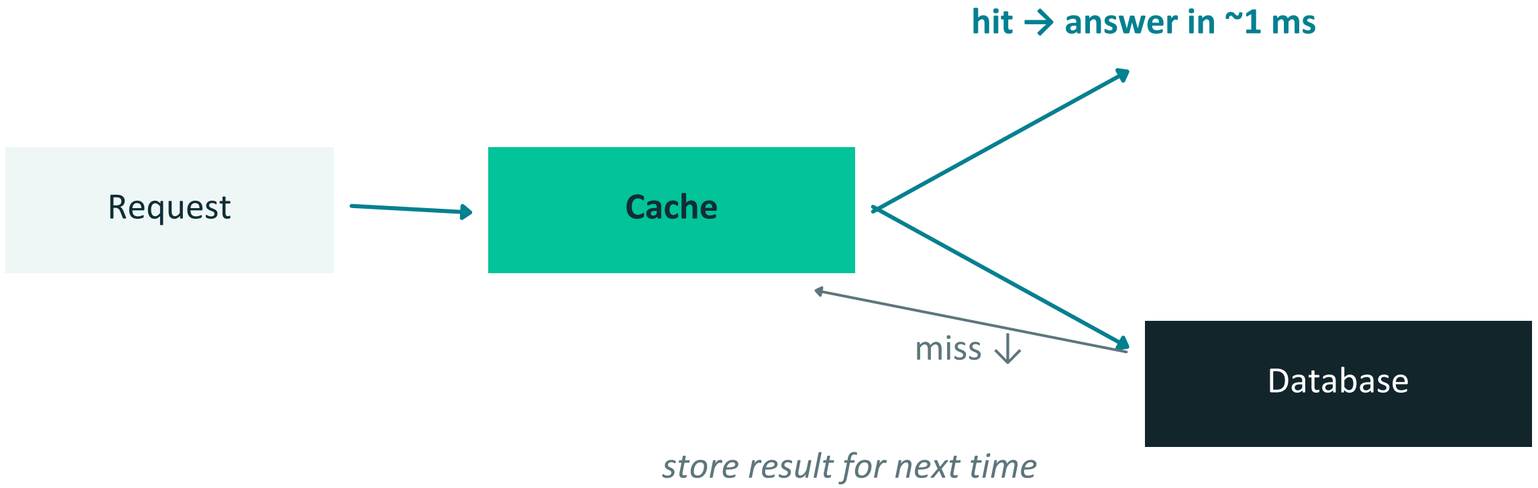
DIAGNOSIS

Why systems actually get slow

1. **Slow database queries**
   missing indexes, N+1 loops
2. **Too many network calls**
   chatty APIs, one call per item
3. **Sequential processing**
   waiting for work that could overlap
4. **Large payloads**
   shipping data nobody asked for
5. **Inefficient algorithms**
   the $O(n^2)$ hiding in a loop
6. **Resource contention**
   everyone queuing for one lock
7. **External dependencies**
   your speed is their speed

Measure before you optimise; the slow part is almost never where you guessed.

Caching: the fastest query is the one you never run



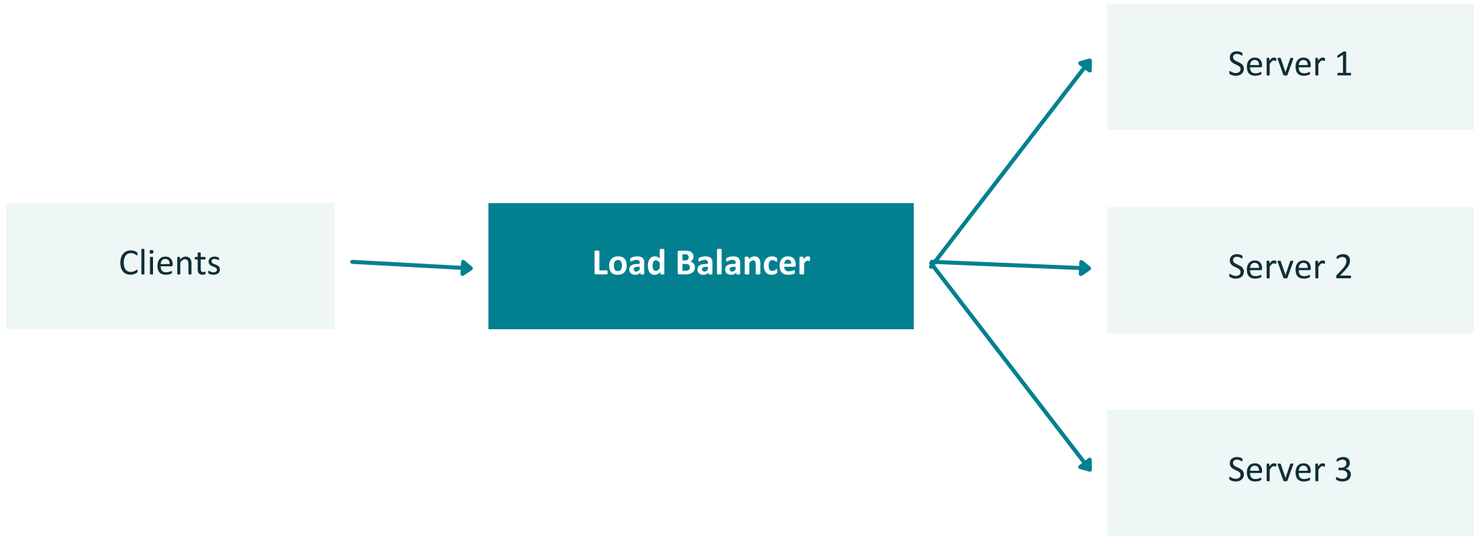
CACHE WHAT?

- Read often
- Changed rarely
- Expensive to compute
- Tolerates being slightly stale

The catch: invalidation: when the real data changes, the cached copy is a lie until you refresh it. Decide the acceptable staleness per data type: menu, 5 minutes; order status, never cache it.

Tools you will meet: Redis, Memcached, HTTP caching headers, CDN caching.

Load balancing: many hands, one front door



spreads requests · health-checks servers · hides failures from users

THE CATCH

This only works if any server can answer any request.

Which is exactly the stateless idea, next slides.

The load balancer is also your first reliability tool: a dead server simply stops getting traffic.

QUALITY 2 · SCALABILITY

What is Scalability of a system?

Two ways to grow: up or out

VERTICAL SCALING: SCALE UP



simple, no code changes
but: a ceiling, a price curve,
and still one point of failure

HORIZONTAL SCALING: SCALE OUT



keep adding machines as load grows
but: needs a load balancer
and stateless design

Cloud pricing made the choice for you: **ten small machines are cheaper than one huge one.**

The server does not remember you

Each request must carry everything the server needs to understand it.

```
// wrong: dies on restart,
// breaks with 2 instances
let loggedInUser = null;

// right: prove it every time
GET /me
Authorization: Bearer <token>
```

CONSEQUENCES YOU WILL MEET

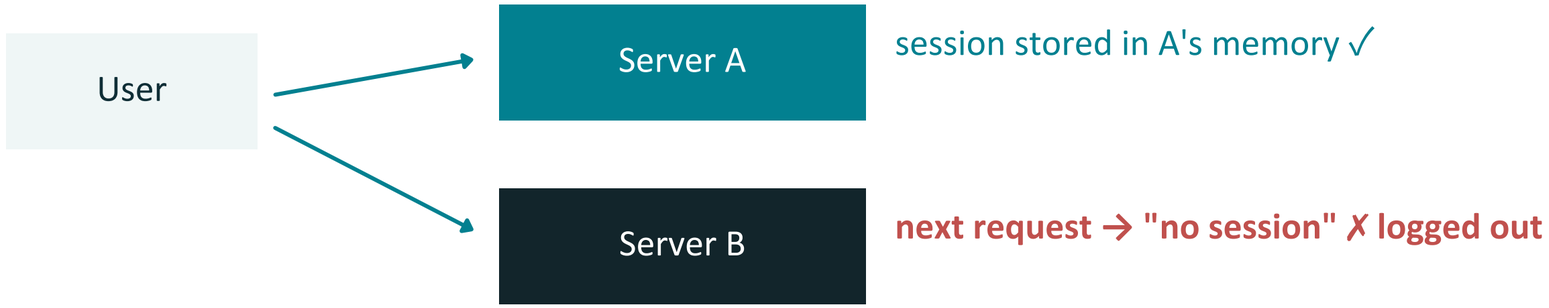
- Your token is sent on every single request
- Any server instance can answer any request
- Nothing user-specific may live in a server variable

The second bullet is the entire secret of horizontal scaling.

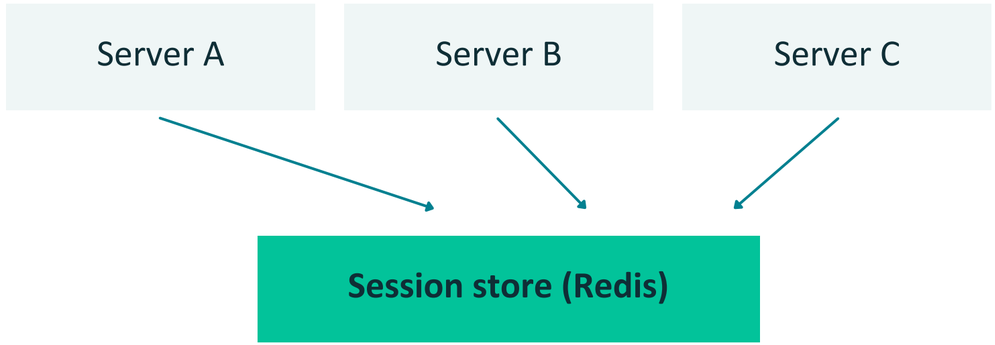
STATEFUL VS STATELESS

What session state does to a scaled system

THE TRAP



FIX A: SHARED SESSION STORE



FIX B: STATELESS TOKENS

```
Authorization: Bearer <JWT>
// the request itself proves
// who you are - any server
// can verify it locally
```

If servers hold no per-user state, any server can answer any request, and scaling out becomes trivial.

EXECUTION SHAPES

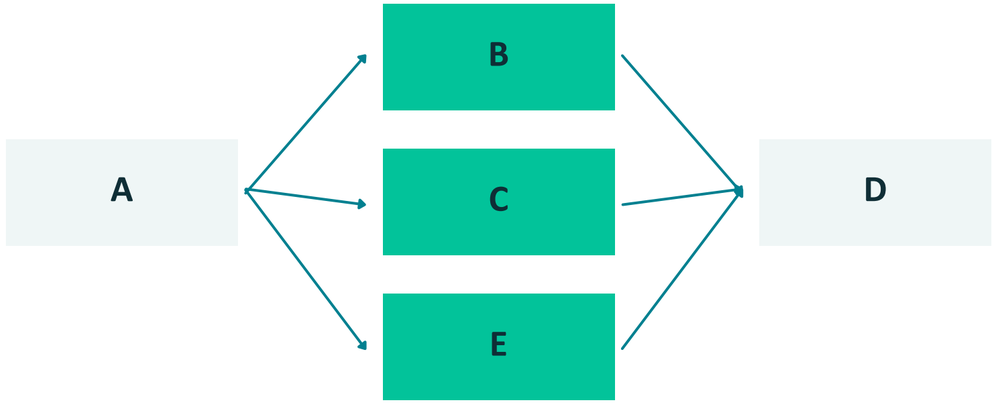
Sequential vs parallel

SEQUENTIAL: ONE AFTER ANOTHER



total time = A + B + C + D (e.g. 3 × 200 ms = 600 ms)

PARALLEL: INDEPENDENT WORK AT THE SAME TIME



total time = slowest branch (200 ms)

PROJECT EXAMPLE

A dashboard needs courses, grades and notices: three independent queries. Fire them together, not in a row: 600 ms becomes 200 ms.

Parallelism is free speed; but only for work that is truly independent.

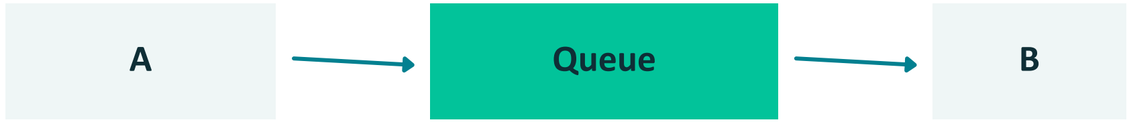
Synchronous vs asynchronous

SYNCHRONOUS: THE CALLER WAITS



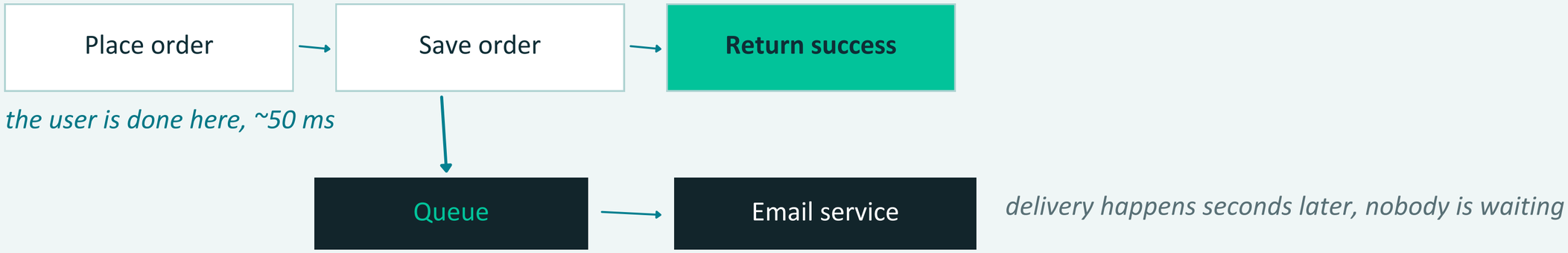
A is blocked until B answers

ASYNCHRONOUS: FIRE AND MOVE ON



A returns immediately; B works later

THE CLASSIC EXAMPLE: EMAIL AFTER AN ORDER



Never make the user wait for work they do not need to see.

QUALITY 3 · RELIABILITY

Reliability, and what "three nines" costs

Reliability: the ability of a system to continue providing its intended service correctly over time, including when components fail.

Each extra nine costs roughly 10× the engineering effort:
redundancy, failover, monitoring, on-call, rehearsed recovery.

AVAILABILITY = UPTIME ÷ TOTAL TIME

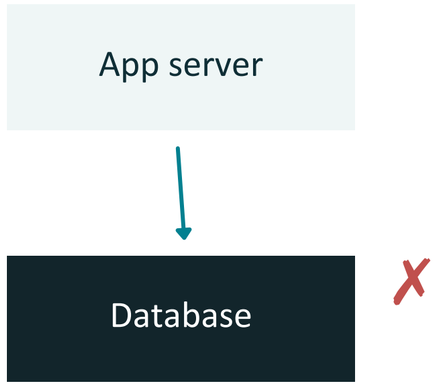
99%	≈ 3.65 days down / year
99.9%	≈ 8.8 hours down / year
99.99%	≈ 53 minutes down / year
99.999%	≈ 5 minutes down / year

Failure is not an exception. At scale, something is always failing; design for it.

THE FIRST QUESTION

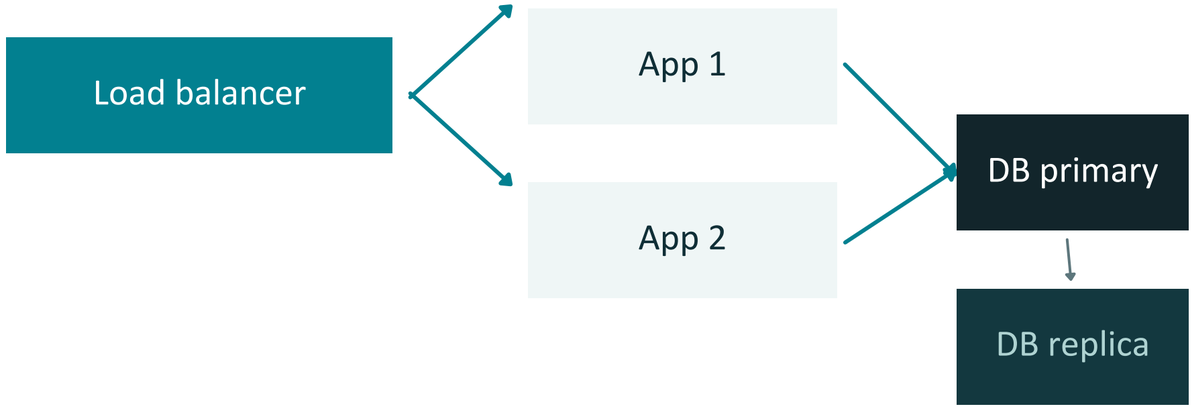
Single points of failure and and redundancy

ONE OF EVERYTHING



DB dies → system dies.
Every single box on this diagram is a single point of failure.

REDUNDANCY: NO SINGLE POINT OF FAILURE



replica follows the primary; takes over if it dies

Ask of every box and every arrow in your design: **"What happens when this dies?"** If the answer is "everything stops", you found a single point of failure.

REAL WORLD · WHEN GIANTS FALL

Two famous outages, two lessons you can use

AWS S3 · FEBRUARY 2017

One mistyped command during routine debugging removed too many servers.

- ≈ 4 hours down in the main US region
- Thousands of sites and apps broke with it
- Even status dashboards depended on S3

Lessons: humans are a failure mode · a shared dependency is a shared SPOF

META · OCTOBER 2021

One network config change withdrew the routes that tell the internet where Facebook lives.

- ≈ 6 hours: Facebook, WhatsApp, Instagram all gone
- Internal tools were on the same network
- Even office badge readers were affected

Lessons: blast radius is a design choice · recovery paths must be independent

Class question: **what is the S3, and what is the BGP, of your food delivery design?**

THE DISTRIBUTED REALITY

Partial failure: B is down, A is not



A is not down. B is down. That is a partial failure, and it is the normal failure mode of any system split across a network.

The question a partial failure forces on you: what should A do right now: wait, retry, give up, or degrade gracefully?

In a monolith, everything fails together. On a network, things fail one at a time.

THREE COPING TOOLS

Timeout · Retry · Circuit breaker

1 Timeout

Do not wait forever.

Give every call a deadline. A request that hangs is worse than one that fails, it holds a thread, a connection, a user.

```
call B, give up after 2 s
```

2 Retry

Try again, carefully.

Great for transient network blips. Only safe on idempotent operations, with a limit and a growing delay between attempts.

```
retry ×3, backoff 1s → 2s → 4s
```

3 Circuit breaker

Stop kicking a dead horse.

After repeated failures, stop calling B. Fail fast to a fallback. Occasionally let one probe through to see if B recovered.

```
closed → open → half-open
```

The circuit breaker pattern comes from Michael Nygard's *Release It!* (2007), the book of production failure stories.

QUALITY 4 · SECURITY

Security is an architectural concern

AUTHENTICATION

"Who are you?"

- Login with password (hashed!)
- Tokens prove identity on every request
- Multi-factor for sensitive systems

AUTHORIZATION

"What are you allowed to do?"

- Roles: student, lecturer, admin
- Checked on the server, every request
- A student must not see others' grades

Security is not something added at the end. It is an architectural concern from day one.

For further readings: XSS, CSRF, SSRF and friends: the OWASP Top 10 list.

SECURITY RULE NUMBER ONE

Never trust the client

IN THE BROWSER

```
// anyone can edit this in devtools
let price = 4999;
if (form.coupon === "STAFF") price = 0;
sendOrder({ total: price });
```

ON THE SERVER

```
// the only version that counts
const item = await repo.findById(id);
const total = pricing.apply(item, user);
order.create({ total });
```

Client-side checks are for user experience. Server-side checks are for correctness and security.

Anything the client sends can be forged: prices, user ids, roles, quantities, hidden fields. Validate on arrival, every time, even if the form already checked it.

PROTECTING DATA

Encrypt in transit, encrypt at rest, and hash passwords

In transit

HTTPS / TLS on every connection. Free certificates (Let's Encrypt) removed the last excuse.

```
client ⇄ 🔒 ⇄ server
```

At rest

Encrypt the database volume and the backups. A stolen disk should be a useless disk.

```
db files 🔒  · backups 🔒
```

Passwords

Never stored, never encrypted, hashed with a slow one-way function: bcrypt or argon2.

```
pw → bcrypt →
$2b$10$N9qo8…
```

Why hash, not encrypt? Encryption has a key, and keys leak. **A hash has no way back.**

THE CLASSIC ATTACK

SQL injection: when input becomes code

VULNERABLE

```
// user input pasted into the query
const q = "SELECT * FROM students " +
  "WHERE name = '" + input + "'";

// input: '; DROP TABLE students;--
// goodbye, students table
```

SAFE

```
// parameterized: input stays data
db.query(
  "SELECT * FROM students WHERE name = ?",
  [input]
);
// input can never become SQL
```



One habit fixes it forever: parameterized queries. Your ORM already does this. Do not bypass it.

CONFIGURATION

Config changes between machines. Code does not.

NEVER DO THIS

```
const db = connect(
  "postgres://admin:P@ss123@localhost/app"
);
```

DO THIS

```
const db = connect(process.env.DATABASE_URL);
// .env locally, real values on the server
```

SECRETS NEVER ENTER GIT

No API keys, no passwords, no tokens. Once a secret is pushed it is public forever. Deleting the file does not delete the history.

IN THE REPOSITORY

```
.gitignore      ->  .env, node_modules
.env.example    ->  committed template
DATABASE_URL=
JWT_SECRET=
```

Almost every student team leaks a key once. Do not be that team.

Technology: the last decision, not the first

Only now, with the domain, the components, the communication style and the quality attributes understood, are we equipped to choose tools.

REUSE

The reuse ladder: four words you will use every week

1

Module

One file of related code. The smallest unit of organisation.

```
utils/dates.js
```

2

Package

Modules bundled with a name and a version so others can install them.

```
npm: date-fns
```

3

Library

Code you call. You stay in control of the flow.

```
axios, requests
```

4

Framework

Code that calls you. It owns the skeleton, you fill the gaps.

```
Express, Spring, Django
```

All four are the same idea at different sizes: do not rewrite what already exists.

THE KEY DISTINCTION

Inversion of control: who calls whom?

LIBRARY: YOUR CODE IS THE MAIN CHARACTER

```
// you decide when to call it
const res = await axios.get(url);
const clean = _.uniq(res.data);
saveToFile(clean);

You call the library.
```

FRAMEWORK: YOUR CODE IS A PLUGIN

```
// the framework decides when
app.get("/courses", (req, res) => {
  res.json(listCourses());
});

Who called this? Express did.
```

A framework gives you structure for free, and takes some freedom in return. That trade is the decision.

A DECISION YOU WILL MAKE IN WEEK 1

SQL or NoSQL, decided by requirements

RELATIONAL (SQL)

- Data has a stable, known shape
- You need joins across entities
- Integrity matters: no orphan registrations
- Transactions across several tables

`Postgres · MySQL · SQLite`

DOCUMENT (NOSQL)

- Shape varies or changes fast
- Documents are read whole, rarely joined
- Schema flexibility beats strict integrity
- Very high write volume

`MongoDB · Firestore · DynamoDB`

Most student projects have relational data. Choose NoSQL because your data is document-shaped, not because it sounds modern.

WORKED EXAMPLE

SQL or NoSQL? Decide for four real data sets

Orders + payments	joins users↔orders↔payments · money must never orphan · transactions	→ **SQL (Postgres)**	*the deciding need: integrity + joins*
Restaurant menus	nested tree, read whole · shape differs per restaurant · rarely joined	→ **Document (MongoDB)**	*the deciding need: flexible shape*
Carts + sessions	tiny, fetched by key · expires in minutes · must be instant	→ **Key-value (Redis)**	*the deciding need: speed by key*
Driver GPS pings	huge append-only stream · queried by recent time window	→ **Wide-column (Cassandra)**	*the deciding need: write volume*

One system, four stores, each earned by a requirement. **You choose per data shape, never "one DB for the system".**

TEAM AGREEMENTS

Conventions are architecture too

1. **Naming**

   One casing style for files, classes, and endpoints. Decide once.

2. **Formatting**

   A formatter like Prettier or Black. Removes an entire class of noisy diffs.

3. **Errors**

   Agree where errors are caught, what the client sees, what gets logged.

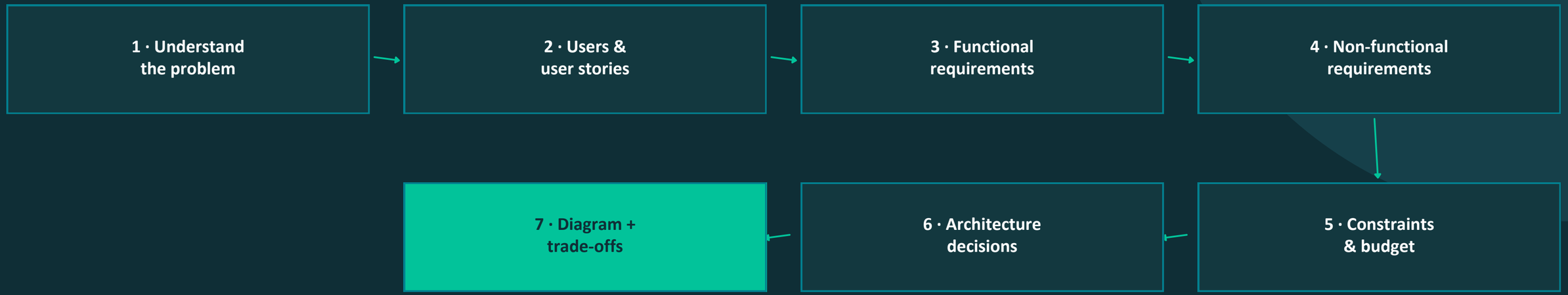
4. **Definition of done**

   Merged, reviewed, tested, documented. Not "works on my laptop".

Agree these in week 1. They cost twenty minutes and save days.

HANDS-ON · THE HEART OF THIS LECTURE

Architect it yourself: three systems, revealed step by step



EX 1 · University registration

EX 2 · Food delivery at scale

EX 3 · Ticket booking rush

USE THIS EVERY TIME

The architecture worksheet: 13 questions per system

1. Who are the users?
2. What are the main user stories?
3. What are the main domain concepts?
4. What are the functional requirements?
5. What are the important NFRs?
6. What are the architectural drivers?
7. What are the major components?
8. Monolith or microservices? Why?
9. How do components communicate?
10. What happens when load increases?
11. What happens when something fails?
12. What security controls are required?
13. **What are the major trade-offs?**

Required answer shape for Q13:

"We chose X because A, accepting B and C."

"We chose microservices" is not an answer. **"We chose X because A, accepting B and C" is.**

Who uses it, and what do they need?

Students

- View courses and free seats
- Register for courses
- Drop courses
- View timetable

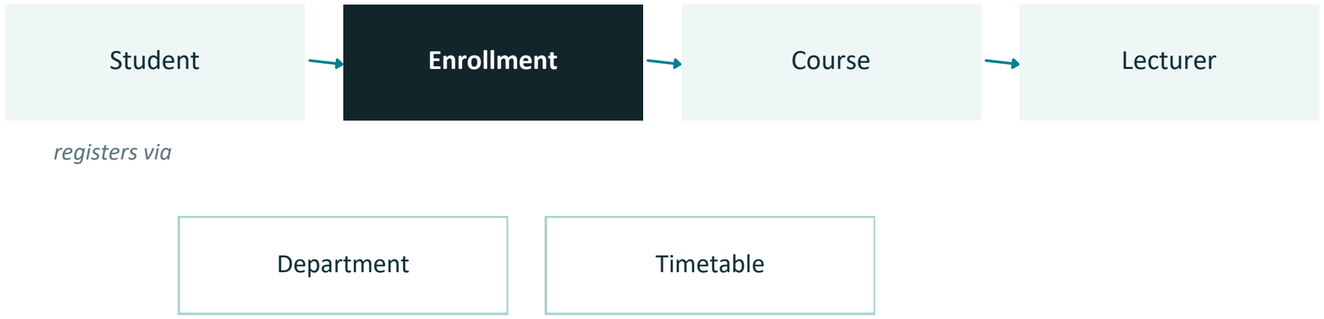
Lecturers

- Create courses
- Set capacity
- View enrolled students

Administrators

- Manage students and courses
- Generate reports

What are the major concepts here?



With only this, what would you build?

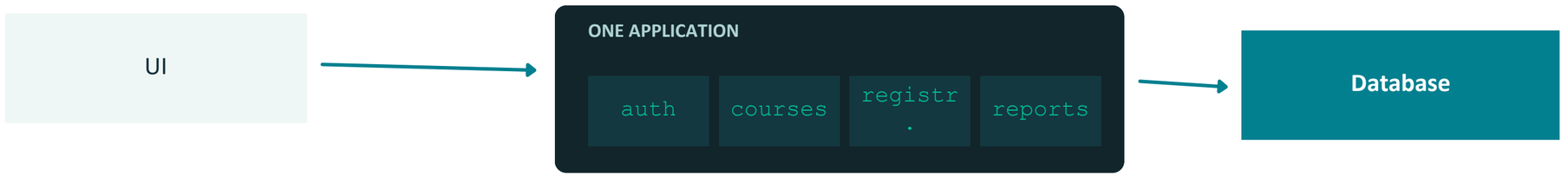
The design step

CAPABILITIES FROM THE STORIES

- **Authentication**
- **Course Management**
- **Registration**
- **Student Management**
- **Reporting**

"Given what we know now, is there any reason NOT to build this simple monolith?"

THE COMMITTED DESIGN · V1



This design is right for the evidence so far. Now watch one new fact attack it.

EXERCISE 1 · STEP 4

9:00 AM changes everything

THE REVEAL · NFRS

- 20,000 students, 2,000 concurrent normally
- Registration opens 9:00 AM for everyone
- Up to 15,000 may hit register at 9:00
- Capacity must NEVER be exceeded
- Under 2 s response · 99.9% availability

is this still just CRUD? ✗ No.

THE SEAT RACE · COURSE CAPACITY 100, ONE SEAT LEFT



THE FIX

Transaction: check + decrement as ONE atomic step

Constraint in the DB: seats >= 0, the last line of defence

Result: one of A or B gets a clean "course full" message

One business rule (never exceed capacity) reached down and **chose transactional SQL for us.**

EXERCISE 1 · STEP 5 · REVEAL: THE BUDGET

Three developers and $500 a month decide the rest

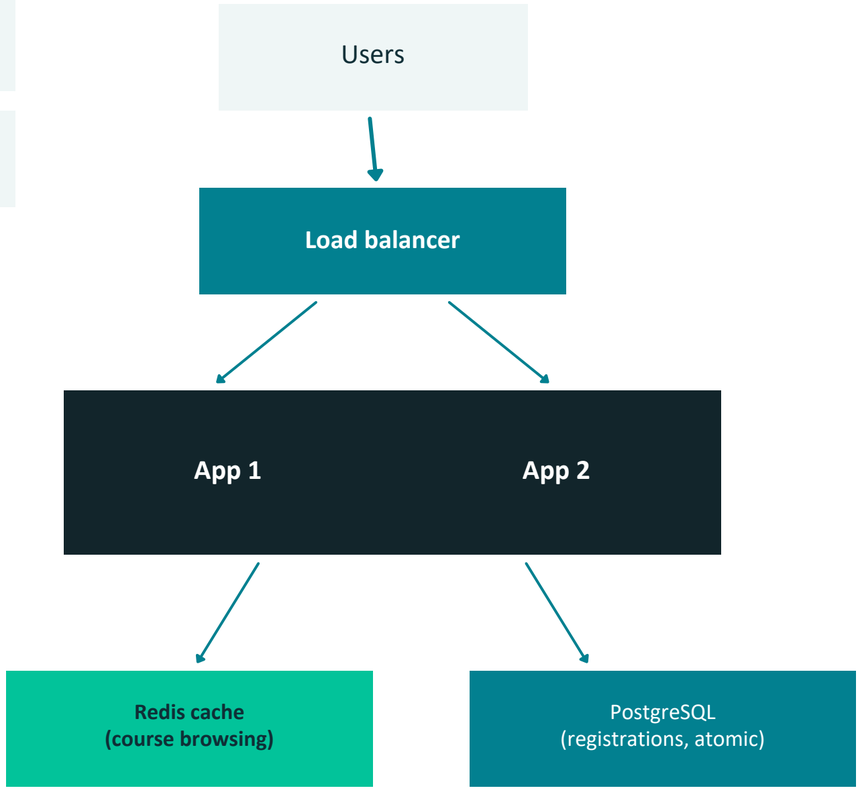
RESOURCES AVAILABLE

2 app servers	**1 PostgreSQL**	**1 Redis**
One team	**no DevOps team**	**$500 / month**

microservices? ✗ No.

- One team : no coordination problem to solve
- Strong transactions: one database is a feature here
- No ops capacity for many deployments
- The 9 AM spike is solved by 2 stateless servers + cache

FINAL ARCHITECTURE



INSIDE EACH APP

```
auth/
students/
courses/
registration/
reports/
```

Good architecture is not the most sophisticated architecture. **It is the one that fits.**

EXERCISE 2 · FOOD DELIVERY AT SCALE · STEP 1

A domain you already know, a company you do not

Customers: search restaurants, browse menus, order, pay, track. Restaurants: manage menus, accept orders, update status. Drivers: accept deliveries, update location, complete.

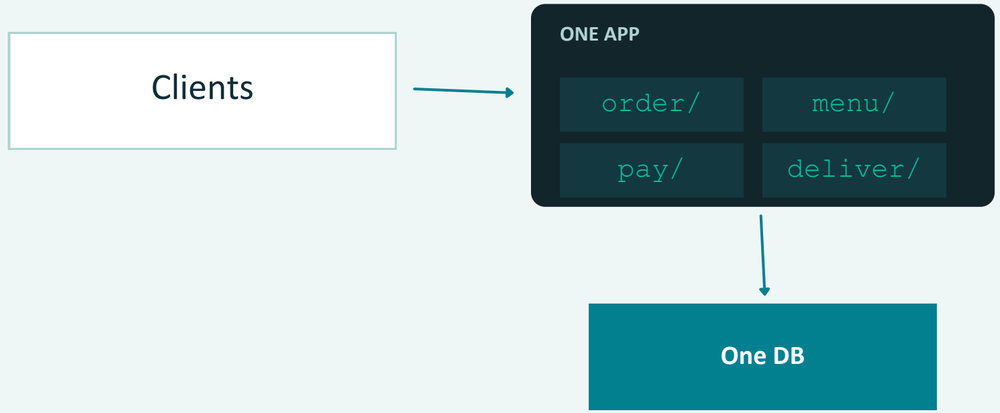
STEP 1 · CAPABILITIES

Customer	**Restaurant**	**Order**
Payment	**Delivery**	**Notification**

one application, or several?

With today's evidence: one modular monolith. Commit to it.

THE COMMITTED DESIGN · V1



exactly the university answer, and rightly so, for now

Reveal nothing yet. Let the monolith answer stand. Then turn the page.

EXERCISE 2 · STEP 2 · REVEAL: THE SCALE

Five million users arrive

THE FACTS

5M	500k	50k	10k	10
registered users	daily active	concurrent	orders / minute peak	cities

Do all parts of the app feel this load the same way?

Restaurant search	huge READS	→	cache + replicas
Ordering	heavy WRITES, lunch spike	→	scale out horizontally
Payment	low volume, must NEVER fail	→	isolate + protect
Delivery tracking	GPS firehose, every few seconds	→	own write pipeline

Four load shapes, one box, one scaling answer. **That mismatch is the real pressure to split.**

EXERCISE 2 · STEPS 3-4 · REVEAL: TEAMS AND MONEY

Five teams, a DevOps crew, $15,000 a month

REVEAL 1 · THE ORGANISATION

Team A **Customer**	**Team B** **Restaurant**	**Team C** **Orders**	**Team D** **Payments**	**Team E** **Delivery**

Conway: the system will mirror this chart. Plan the mirror instead of fighting it.

REVEAL 2 · THE RESOURCES

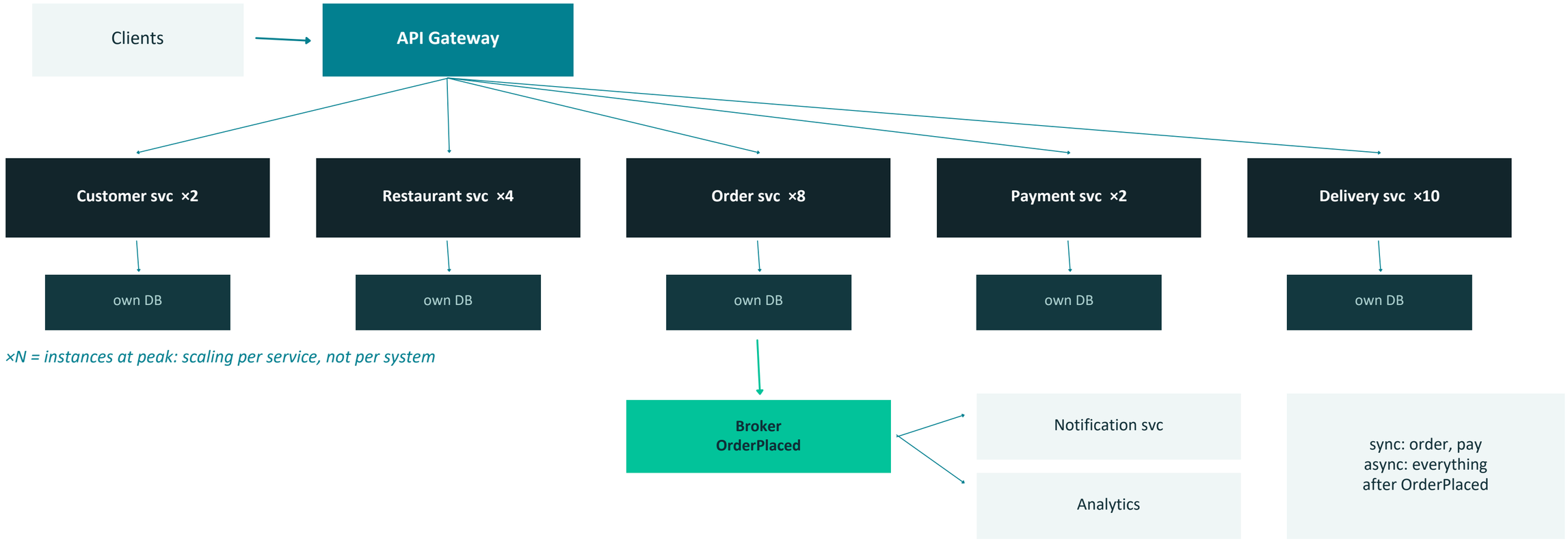
20 developers	**cloud infra**	**dedicated DevOps**
CI/CD pipelines	**monitoring**	**$15,000 / month**

RUN THE CHECKLIST

- Several teams blocking each other ✓
- Load shapes needing different scaling ✓
- Deploy independence wanted ✓
- Ops maturity to run it ✓

Every checklist box ticks for the first time today. **Decision: microservices, said with reasons.**

Services, gateway, events, and per-service scaling



API Gateway

DB per service

Events via broker

Independent scaling

Nobody memorised these patterns. The problem demanded every one of them.

EXERCISE 3 · TICKET BOOKING · STEPS 1-2

A quiet system, then one tweet

THE SYSTEM · QUIET DAYS

Concerts: 50,000 seats. Search events, view seats, select, reserve, pay, receive tickets.
Load: about 1,000 users per day.



← the whole v1 design. Correct.

THE REVEAL · ONE TWEET

A global star announces ONE show.
Tickets open 10:00 AM.

10:00:01 → 500,000 users on the site at once

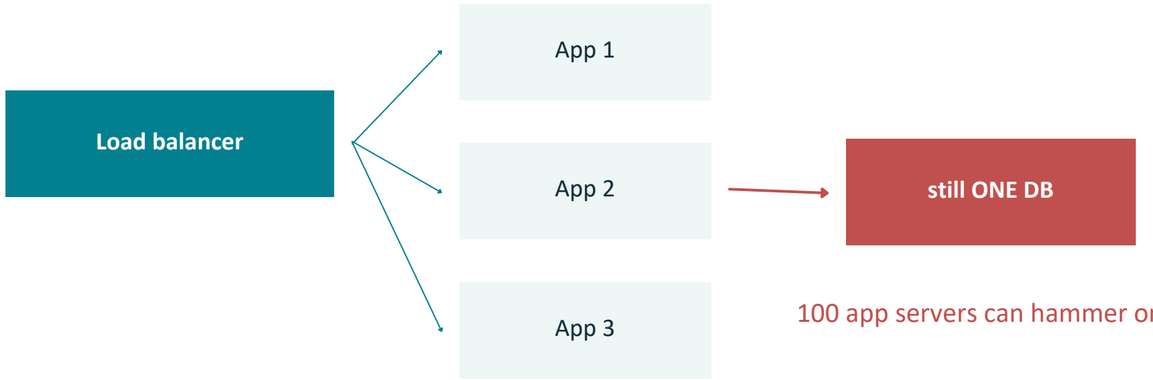
What fails FIRST?

Same system, new number, dead design. Numbers are architecture.

EXERCISE 3 · STEPS 3-4 · SCALE OUT, CACHE THE READS

Can we just add 100 servers? No: the database is still one

STEP 1 · SCALE OUT (NECESSARY, NOT SUFFICIENT)



100 app servers can hammer one database even harder

STEP 2 · LOOK AT WHAT THEY READ

500,000 people are loading the SAME event page and the SAME seat map.

1 cache entry → 500,000 requests served
DB reads collapse to almost nothing

THE CAVEAT THAT MATTERS

A seat map 5 seconds stale is fine to LOOK at. It is never fine to SELL from. Browsing reads come from cache; the purchase decision happens only inside the database transaction.

Cache the browsing. Never the buying.

EXERCISE 3 · STEPS 5-6 · THE LAST SEAT, AND A FAILING PAYMENT

Concurrency and the reservation state machine

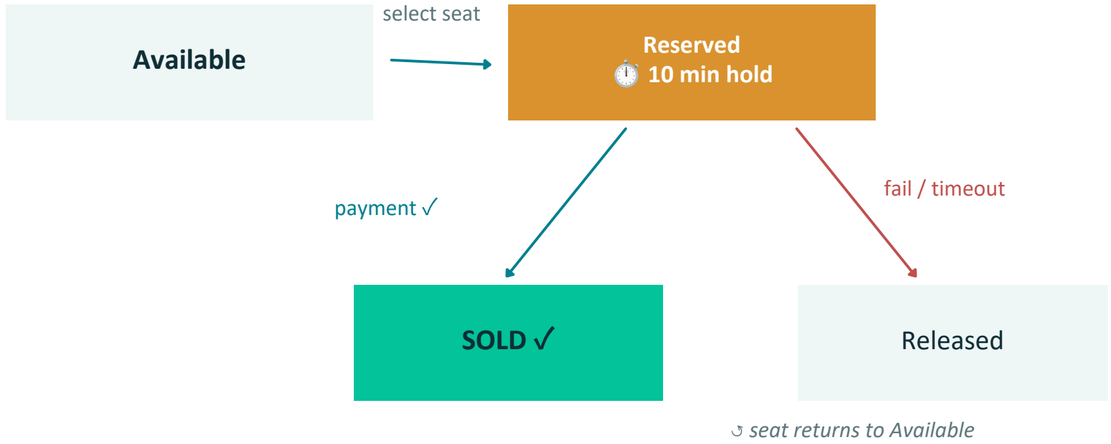
PROBLEM 1 · THE LAST-SEAT RACE (YOU KNOW THIS ONE)

Buyer A and buyer B, same seat, same millisecond → atomic check + write in one transaction, constraint as backstop → exactly one winner, one clean "sold out".

PROBLEM 2 · THE WINNER STILL HAS TO PAY

Payment takes a minute and can fail. Mark the seat sold too early: failed payments strand seats. Never hold it: someone else buys it mid-payment.

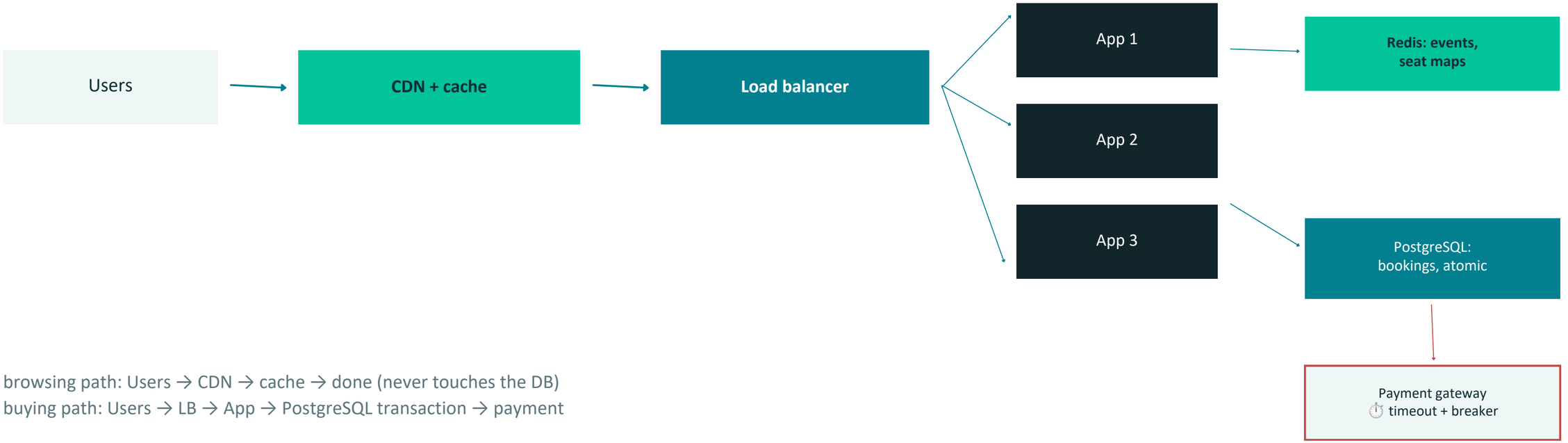
THE FIX · A STATE MACHINE WITH A CLOCK



Every arrow is a business rule the system enforces: who may move a seat between states, and when. Design the arrows and the code writes itself.

Architecture must handle concurrency and time, not just boxes and lines.

One problem taught seven concepts: the final diagram



browsing path: Users → CDN → cache → done (never touches the DB)
buying path: Users → LB → App → PostgreSQL transaction → payment

Performance → **Scalability** → **Load balancing** → **Caching** → **Concurrency** → **Transactions** → **Reliability**

section 6, compressed into one story

Every box on this diagram has a birthday: the exact moment in the story that forced it.

7

Putting It All Together

Architect an online food delivery system

THE BRIEF

Design an online food delivery system

FUNCTIONAL REQUIREMENTS

- Customer can browse restaurants and menus
- Customer can place an order
- Customer can pay online
- Restaurant receives and confirms the order
- Driver delivers; customer tracks the order

NON-FUNCTIONAL REQUIREMENTS

- 100,000 registered users
- Response time under 2 seconds
- 99.9% availability
- Payments must be secure

Eight steps from this brief to an architecture. Every step is one section of today.

STEPS 1–2 · PROBLEM & FUNCTIONAL REQUIREMENTS

Understand the domain, list the actions

STEP 1: MAJOR CONCEPTS

- Customer
- Restaurant
- Driver
- Order
- Payment
- Delivery

STEP 2: MAJOR ACTIONS

- Browse
- Order
- Pay
- Deliver
- Track

Nothing on this slide mentions technology. That is on purpose, and by now, expected.

Concepts + actions = the raw material for components and APIs.

STEP 3 · ARCHITECTURAL DRIVERS

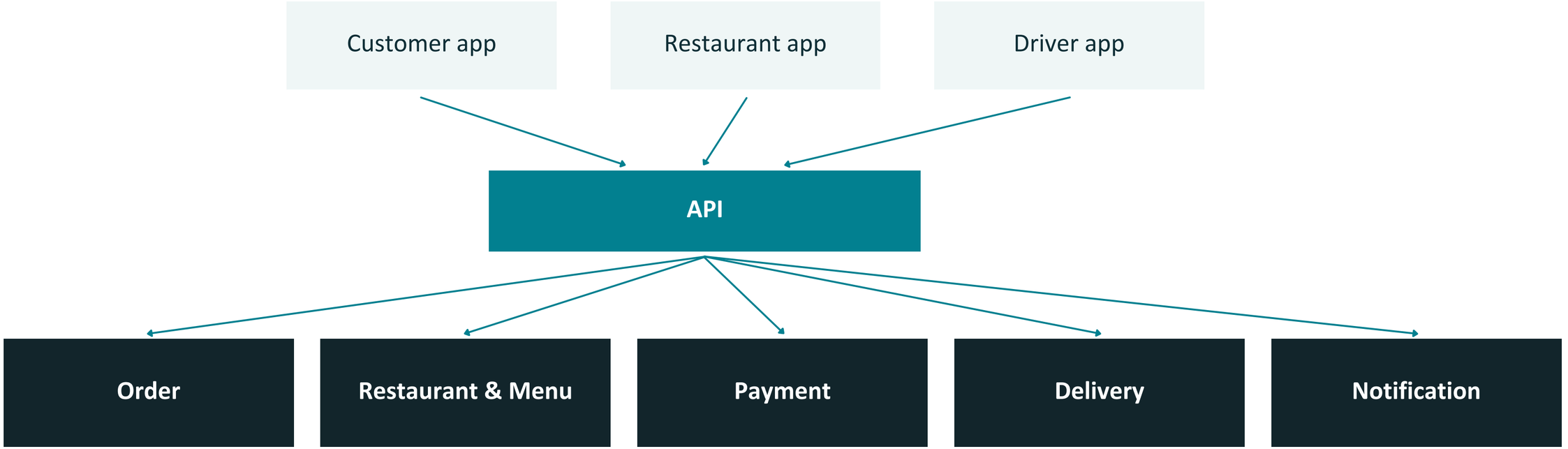
Which requirements will shape the structure?

100,000 users	**SCALABILITY**	load balancer · stateless app servers · caching
Response < 2 seconds	**PERFORMANCE**	cache hot reads · async for slow work · parallel queries
99.9% availability	**RELIABILITY**	no single point of failure · replicas · timeouts & fallbacks
Secure payment	**SECURITY**	HTTPS · never trust client · delegate card data to a payment provider

Requirement → quality attribute → concrete structure. This chain is your report's "justification" section.

STEP 4 · COMPONENTS

From domain concepts to system components



each component: clear responsibility, named after the domain, owns its own data

Components mirror the domain, exactly the promise of section 3.

STEP 5 · THE STYLE DECISION

Do we actually need microservices?

INTERROGATE THE DRIVERS

- 100,000 registered users, how many requests per second is that, at peak?
- One team of four, or many teams?
- Does any component need to scale or deploy independently today?
- What does the network cost us if we split?

The marks are in the justification, not the diagram.

STEP 6 · COMMUNICATION

What must be synchronous, and what must not

SYNCHRONOUS: THE CUSTOMER IS WAITING

- Browse restaurants & menus
- Place the order
- Authorize the payment

these block the response, keep them fast

ASYNCHRONOUS: NOBODY IS WAITING

- Email the receipt
- Notify the restaurant
- Update analytics
- Assign a driver

these go through the queue

Rule from section 6: never make the user wait for work they do not need to see.

STEPS 7–8 · PERFORMANCE & RELIABILITY

Add the quality machinery, and ask the failure question

STEP 7: THE PERFORMANCE ADDITIONS

- Load balancer in front of 2+ stateless app instances
- Cache for menus and restaurant lists (read-often, changed-rarely)
- Queue for all async work from step 6
- DB replica: reads scale, and reliability improves for free

STEP 8: THE FAILURE DRILL

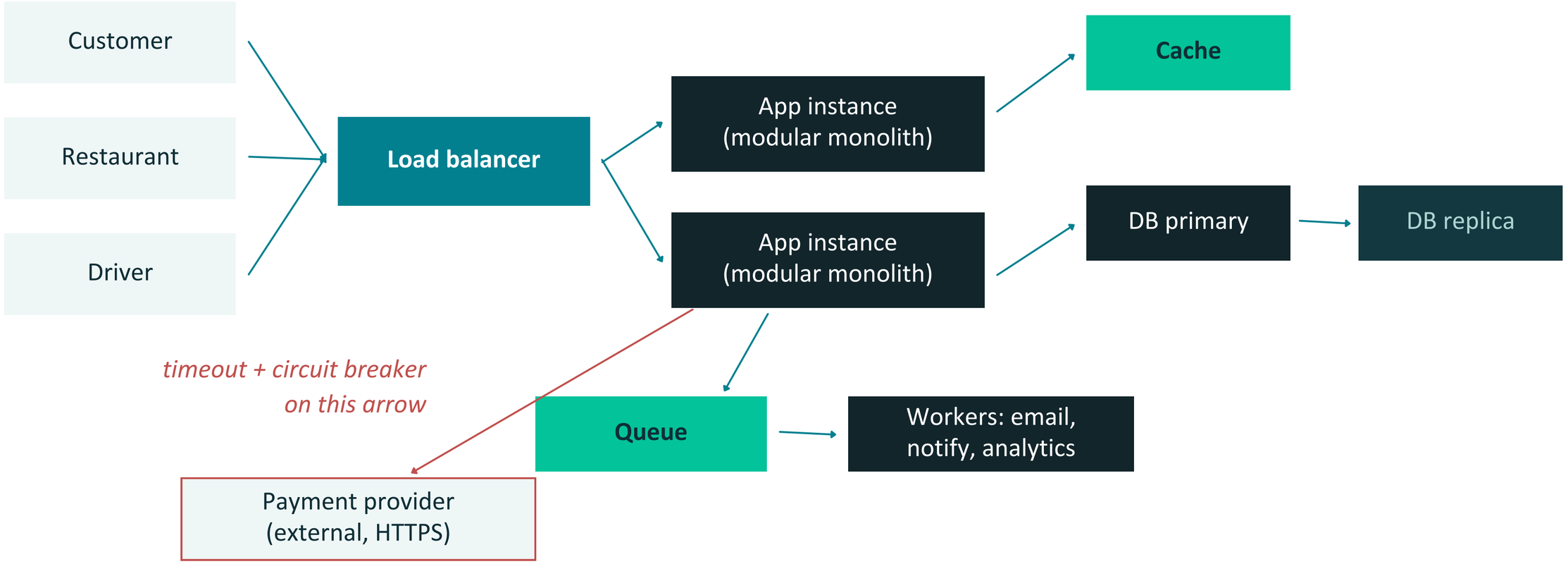
"What happens if the payment service goes down?"

- Timeout: fail in 2 s, not never
- Circuit breaker: stop hammering it
- Degrade: save order as pending-payment, tell the user honestly

Ask the failure question of every arrow in your final diagram.

THE RESULT

One defensible architecture for the brief

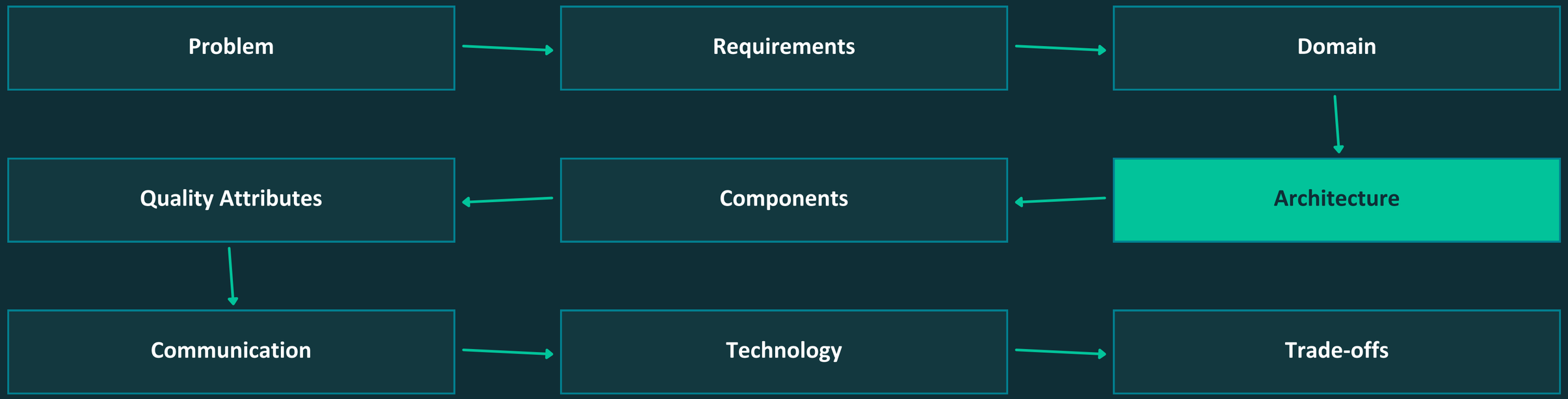


Every box exists because a driver demanded it. Nothing here is decoration.

This is just one defensible answer

TAKE THIS TO YOUR PROJECT

How to architect a system



Problem first. Requirements split into functional and non-functional. The domain gives you components; the drivers give you structure. Choose communication styles, choose technology last, and write down every trade-off you accepted.

BEFORE YOU DRAW A SINGLE BOX

The seven-question checklist

1. What problem are we solving?
2. What are the functional requirements?
3. What are the important non-functional requirements?
4. What are the major concepts and components in the domain?
5. How should those components communicate?
6. What happens when the system gets slow, overloaded, or fails?
7. **Why did we choose this architecture over the alternatives?**

Answering these seven well is worth more than knowing thirty architecture patterns.

WHERE TO GO NEXT

Further reading and tools

BOOKS & ARTICLES

- Clean Architecture: Robert C. Martin
- Domain-Driven Design: Eric Evans (2003)
- Release It!: Michael Nygard
- Fowler: "MonolithFirst" & the Microservices Guide
- The Twelve-Factor App: 12factor.net
- Conway, "How Do Committees Invent?" (1968)

CONCEPTS WE ONLY NAMED

- Bounded contexts, aggregates, domain events
- Hexagonal & clean architecture
- Sagas & eventual consistency
- Distributed tracing
- CAP theorem, CQRS, event sourcing: later courses
- OWASP Top 10 (XSS, CSRF, …)

TOOLS WORTH TEN MINUTES

- Postman / Thunder Client for APIs
- Swagger & OpenAPI for contracts
- Prettier, ESLint, Black for conventions
- GitHub Actions for CI
- draw.io / Excalidraw for diagrams

Materials pack: activity brief · decision record template · API contract template · sample code walkthrough.

Questions?